



**Universidad
Tecnológica
del Perú**

Monografía Proyecto Palant-Pedidos

Curso Integrador I: Sistemas Software

Sección: 32424

Integrantes:

- **U22321849- Caso Barzola Brando Kenny**

Profesora:

Vanessa Oswaldo Salvatierra

CAPÍTULO 1 – ASPECTOS GENERALES DEL PROYECTO	4
1.1. Información de la empresa	4
1.1.1. Descripción general	4
1.1.2. Visión	4
1.1.3. Misión	4
1.1.4. Matriz FODA	5
1.1.5. Organigrama	5
1.1.6. Objetivos estratégicos	6
1.1.7. Entorno y situación del sector	6
1.1.8. Planes y proyecciones de la empresa	7
1.2. Definición del problema	7
1.2.1. Identificación del problema	8
1.2.2. Situación actual y necesidades	8
1.2.3. Alternativas de solución	8
1.2.4. Justificación de la solución elegida	9
1.3. Objetivos del proyecto	10
1.3.1. Objetivo general	10
1.3.2. Objetivos específicos	10
1.4. Alcances y limitaciones del proyecto	11
1.4.1. Alcances funcionales y técnicos	11
1.4.2. Limitaciones del proyecto	11
1.5. Requisitos del sistema	12
1.5.1. Requisitos funcionales	12
1.5.2. Requisitos no funcionales	12
1.6. Modelo de negocio	13
1.7. Análisis comparativo de soluciones tecnológicas	14
1.8. Planificación del proyecto	15
1.8.1. Metodología de desarrollo	15
1.8.2. Project Charter	16
1.8.3. Estructura de desglose del trabajo (WBS)	17
1.8.4. Cronograma y diagrama de Gantt	17
1.8.5. Asignación de roles, hitos y entregables	18
1.8.6. Herramientas de gestión utilizadas	19

CAPÍTULO 2 – MARCO TEÓRICO	21
2.1. Fundamentos del desarrollo de software.....	21
2.2. Análisis y diseño orientado a objetos	22
2.3. Modelado de procesos de negocio (BPM)	26
CAPÍTULO 3 – DISEÑO DEL SISTEMA	34
3.1. Diseño de interfaz de usuario (UX/UI)	34
CAPÍTULO 4 – DESARROLLO DE LA SOLUCIÓN	41
4.1. Arquitectura del sistema	41
4.2. Desarrollo iterativo	44
4.3. Uso de librerías Java	45
4.4. Control de versiones	49

CAPÍTULO 1 – ASPECTOS GENERALES DEL PROYECTO

1.1. Información de la empresa

1.1.1. Descripción general

PALANT S.A.C. es una empresa peruana líder en el sector textil con más de 25 años de trayectoria, especializada en el diseño, confección y comercialización de indumentaria deportiva de alto rendimiento. La organización se destaca por su dominio en la técnica de sublimación digital y su sólida presencia en el fútbol profesional, siendo el proveedor oficial y auspiciador de diversos clubes en la Liga 1, Liga 2 y Copa Perú, así como de academias de alto rendimiento a nivel nacional.

1.1.2. Visión

Ocupar uno de los primeros lugares en el rubro de la ropa deportiva a nivel Nacional, mejorando la calidad de nuestros productos. Todo ello respaldado en una cultura de excelencia operativa, prontitud de respuesta, así como un alto nivel de atención a nuestros clientes; por medio de personal capacitado y motivado que nos permita tener un crecimiento con rentabilidad sostenida, manteniendo siempre un espíritu de responsabilidad social y respeto al medio ambiente.

1.1.3. Misión

Somos una empresa textil con líneas de negocio orientada en ropa deportiva. Trabajamos para satisfacer los estándares de calidad de nuestros clientes, basados en **capacidad innovadora, flexibilidad y vocación de servicio**, a través de productos diferenciados, logrando así el crecimiento y desarrollo económico de la empresa

1.1.4. Matriz FODA

FODA Estratégico: PALANT S.A.C.			
FORTALEZAS	Dominio de Especialización: Experiencia consolidada en la técnica de sublimación digital para indumentaria deportiva de alta competencia (Liga 1, 2 y Copa Perú).	OPORTUNIDADES	Estandarización de Flujos de Trabajo: Implementar una metodología digital para replicar modelos de éxito en pedidos de provincias con alta demanda.
	Capacidad Instalada: Infraestructura y maquinaria propia que permite autonomía total en la fabricación de pedidos personalizados.		Optimización de Tiempos de Respuesta: Ventaja competitiva frente a empresas del sector que sufren retrasos por falta de control administrativo.
	Cartera de Clientes Fidelizada: Relaciones estratégicas vigentes con clubes y academias regionales que garantizan una demanda cíclica.		Trazabilidad del Producto: Posibilidad de ofrecer al cliente información en tiempo real sobre el estado de su pedido, aumentando la confianza en la marca.
	Versatilidad de Producción: Capacidad para alternar entre la fabricación de grandes lotes institucionales y pedidos específicos a medida.		
DEBILIDADES	Vulnerabilidad en la Integridad de Datos: Alta dependencia de registros manuales y transcripciones físicas, lo que eleva el riesgo de errores en tallas y especificaciones.	AMENAZAS	Brecha Tecnológica en el Sector: Competidores que adoptan herramientas ERP/CRM, desplazando a las empresas con gestión tradicional de pedidos.
	Inexistencia de Monitoreo de Procesos: Falta de un panel de control centralizado para visualizar en qué etapa (Diseño, Sublimación o Confección) se encuentra cada orden.		Inestabilidad de Costos en Insumos: Fluctuación en el precio de materia prima crítica (telas, tintas de sublimación) que afecta el presupuesto de pedidos ya pactados.
	Fragmentación de la Información Operativa: Los detalles técnicos de los pedidos se encuentran dispersos en múltiples canales informales, dificultando la coordinación entre el vendedor y el taller.		Exigencia del Mercado Profesional: Clubes deportivos de alto nivel que requieren mayor formalidad, rapidez y precisión en la entrega de sus <u>equipaciones</u> .
	Limitación en la Planificación de la Producción: Dificultad para calcular fechas de entrega precisas debido a la falta de una visión clara de la carga de trabajo actual.		Exigencia del Mercado Profesional: Clubes deportivos de alto nivel que requieren mayor formalidad, rapidez y precisión en la entrega de sus <u>equipaciones</u> .

El FODA de PALANT S.A.C. contrasta su excelente capacidad técnica en indumentaria con la urgencia de superar su gestión operativa manual.

1.1.5. Organigrama

ORGANIGRAMA ESTRUCTURAL DE LA EMPRESA PALANT S.A.C.



El organigrama funcional de PALANT S.A.C. muestra una estructura liderada por la Gerencia General, dividida en tres áreas clave: Producción, Administración y Comercial.

1.1.6. Objetivos estratégicos

- **Tecnológico/Operativo:** Estandarizar y digitalizar el flujo de trabajo de producción para reducir los errores derivados de registros manuales y optimizar los tiempos de respuesta.
- **Comercial:** Consolidar la cartera de clientes de alto nivel (Liga 1, 2 y Copa Perú) ofreciendo un servicio más formal que incluya la trazabilidad de sus pedidos en tiempo real.
- **Crecimiento:** Replicar los modelos de éxito actuales para captar y gestionar eficientemente un mayor volumen de pedidos provenientes de provincias.

1.1.7. Entorno y situación del sector

Actualmente, **Palant S.A.C.** opera de manera tradicional y empírica en su gestión administrativa. La recepción de pedidos se realiza por canales informales (WhatsApp, llamadas, presencial) y el flujo de información hacia el taller de producción es **netamente manual**.

- **Gestión de Datos:** Se utilizan cuadernos de apuntes y hojas sueltas que circulan por las áreas de diseño y confección.
- **Comunicación Interna:** No existe un método de consulta en tiempo real; para saber el estado de un pedido, el vendedor debe acudir físicamente al taller e interrumpir los procesos.
- **Tecnología:** Existe maquinaria avanzada para la fabricación, pero un vacío tecnológico en la gestión de la información.
- **Planificación y Tiempos de Entrega:** La falta de visibilidad global sobre la carga de trabajo actual en el taller dificulta el cálculo de fechas de entrega exactas, generando un riesgo constante de cuellos de botella y retrasos.

1.1.8. Planes y proyecciones de la empresa

Para alcanzar sus objetivos estratégicos y superar las limitaciones de su entorno actual, PALANT S.A.C. ha trazado un plan de acción enfocado en la ejecución de los siguientes proyectos:

- **Corto Plazo (Desarrollo e Implementación):** Construcción y despliegue del sistema informático **PALANT-PEDIDOS**, incluyendo su integración con una base de datos en la nube. Esto reemplazará inmediatamente el uso de canales informales y registros físicos.
- **Mediano Plazo (Transición Operativa):** Capacitación integral del personal administrativo y de taller para la adopción del nuevo software, asegurando una transición fluida hacia un modelo de trabajo 100% digital y sin cuellos de botella.
- **Largo Plazo (Escalabilidad Comercial):** Consolidada la infraestructura tecnológica interna, la empresa ejecutará un plan de expansión hacia provincias clave, apalancándose en el nuevo sistema para gestionar un alto volumen de ventas a nivel nacional manteniendo los estándares del mercado profesional.

1.2. Definición del problema

En PALANT S.A.C., el problema central radica en la carencia de un sistema de información centralizado para la gestión operativa de los pedidos. Esta deficiencia provoca la fragmentación de los datos técnicos entre las áreas comercial y de producción, lo que se traduce directamente en errores de fabricación (tallas, dorsales o especificaciones incorrectas), retrasos en las entregas por falta de trazabilidad y un incremento en los costos operativos derivados del reproceso de prendas.

1.2.1. Identificación del problema

El problema se materializa en el uso de formatos de papel llenados a mano que circulan por las distintas áreas del taller. Durante la transición de la orden (del vendedor al diseñador, y finalmente a los costureros), factores como la mala caligrafía, el desorden visual de los datos y la falta de estandarización provocan que se fabriquen prendas con características equivocadas, generando cuellos de botella operativos y desperdicio de material.

1.2.2. Situación actual y necesidades

- **Situación actual:** Actualmente, tanto el área de diseño como el área de confección invierten tiempo valioso intentando descifrar hojas de pedido confusas y desorganizadas. La información llega fragmentada y propensa a malas interpretaciones, obligando al personal a pausar sus labores para confirmar especificaciones básicas de las prendas.
- **Necesidades:** La empresa necesita con urgencia reemplazar las hojas manuales por un formato digital como físico que estandarice visualmente la información. Se requiere que especificaciones como tallas, colores y nombres estén ordenadas de forma intuitiva, permitiendo que tanto diseñadores como costureros puedan leer y ejecutar la orden de trabajo sin margen de error.

1.2.3. Alternativas de solución

Alternativa A: Adquisición de un Sistema de Gestión de Manufactura (MES)

Comercial.

- **Descripción:** Implementar un software empaquetado enfocado en la industria textil internacional.
- **Desventaja:** Estos sistemas son rígidos y están diseñados para la producción en masa de lotes idénticos. No se adaptan a la "personalización extrema" (sublimación

de nombres, números y colores únicos por prenda) que requiere PALANT S.A.C. Además, sus interfaces genéricas no resolverían el problema visual para los costureros y exigirían una costosa infraestructura de servidores locales.

Alternativa B: Arquitectura Web basada en Microservicios y PWA.

- **Descripción:** Desarrollar una solución web distribuida alojada en la nube, utilizando contenedores (Docker) y múltiples microservicios.
- **Desventaja:** Representa una sobrecarga tecnológica (*over-engineering*) que añade una complejidad de mantenimiento innecesaria para una PYME. Además, la posible latencia de una aplicación web en el entorno del taller —donde se requiere alta velocidad para procesar órdenes con archivos gráficos— la hace menos viable operativa y económicamente.

Alternativa C: Desarrollo de un Sistema a Medida (PALANT-PEDIDOS).

- **Descripción:** Crear una aplicación de escritorio nativa (JavaFX) conectada a una base de datos en la nube (Supabase), diseñada de cero para el flujo de trabajo exacto de la empresa.
- **Ventaja:** Es la solución más factible y directa. Al ser a medida, permite diseñar interfaces visualmente organizadas e intuitivas que reemplazan las hojas manuales, facilitando la lectura exacta de tallas y especificaciones tanto para el diseñador como para los costureros. Además, ofrece un control centralizado sin costos excesivos de infraestructura.

1.2.4. Justificación de la solución elegida

Se selecciona la **Alternativa C (PALANT-PEDIDOS)** por ser la opción más viable a nivel operativo, técnico y económico. Al ser un desarrollo a medida estructurado con JavaFX y

Supabase, permite diseñar interfaces visuales limpias que reemplazan las confusas hojas manuales, eliminando los errores de transcripción en el taller. Además, garantiza una alta velocidad de respuesta y centraliza la información en tiempo real, evitando los altos costos de licencias de un sistema comercial y el mantenimiento innecesario de una arquitectura web compleja, adaptándose perfectamente a la realidad de la empresa.

1.3. Objetivos del proyecto

1.3.1. Objetivo general

Desarrollar e implementar un sistema de escritorio a medida (PALANT-PEDIDOS) utilizando JavaFX y Supabase, para centralizar y digitalizar la gestión de órdenes de trabajo en PALANT S.A.C., con el fin de eliminar los errores de transcripción manual y optimizar la comunicación entre las áreas de ventas, diseño y confección.

1.3.2. Objetivos específicos

- Diseñar interfaces visuales intuitivas y estandarizadas que reemplacen las hojas de pedido físicas, permitiendo a los diseñadores y costureros visualizar de forma clara y sin margen de error las especificaciones críticas (tallas, nombres a sublimar y colores).
- Integrar una base de datos en la nube (Supabase) para sincronizar la información operativa, garantizando que los datos ingresados por ventas estén disponibles instantáneamente en el taller.
- Implementar un sistema de trazabilidad de estados que permita monitorear en tiempo real la fase exacta en la que se encuentra cada pedido (recepción, diseño, sublimación, confección o entrega), evitando interrupciones físicas en el taller.
- Reducir los tiempos muertos y mermas de producción generados actualmente por la interpretación de registros desorganizados, mejorando la eficiencia general del ciclo de manufactura.

1.4. Alcances y limitaciones del proyecto

1.4.1. Alcances funcionales y técnicos

A. Alcances funcionales:

- **Módulo de Recepción:** Ingreso digital de pedidos con campos estandarizados para especificaciones técnicas (tallas, dorsales, colores, tipo de tela y cantidad).
- **Panel de Taller (Visualización):** Interfaz gráfica optimizada para que el área de diseño y confección puedan leer las órdenes sin ambigüedades.
- **Trazabilidad de Estados:** Capacidad de actualizar y visualizar la fase exacta del pedido (ej. Recepción, Diseño, Sublimación, Confección, Terminado).

B. Alcances técnicos:

- Desarrollo de una aplicación de escritorio nativa utilizando el framework JavaFX, garantizando interfaces fluidas.
- Integración con una base de datos relacional alojada en la nube mediante Supabase, permitiendo la consulta concurrente de información entre los vendedores y el taller.

1.4.2. Limitaciones del proyecto

- **Plataforma:** El sistema PALANT-PEDIDOS está diseñado exclusivamente como una aplicación de escritorio para entornos de trabajo fijos (PCs en tienda y taller). En esta etapa, no se contempla el desarrollo de una versión móvil (App para Android/iOS).
- **Dependencia de Conectividad:** Al utilizar una base de datos centralizada en la nube (Supabase) para asegurar la sincronización en tiempo real, el sistema requiere de una conexión a internet estable y permanente en las instalaciones del taller para operar correctamente.

- Límites Operativos: El alcance del software se centra estrictamente en el flujo de producción y gestión de órdenes de trabajo. No incluirá módulos de facturación electrónica (SUNAT), gestión contable o cálculo de planillas de pago para los operarios.

1.5. Requisitos del sistema

1.5.1. Requisitos funcionales

- **RF1:** El sistema debe permitir el registro, modificación y consulta de un pedido, abarcando los datos del cliente, equipo, fecha pactada de entrega y el detalle específico de cada prenda (talla, número y nombre a sublimar).
- **RF2:** El sistema debe permitir adjuntar y visualizar archivos gráficos (documentos PDF o imágenes) como anexos técnicos a cada orden de trabajo.
- **RF3:** El sistema debe permitir la carga y visualización en pantalla del *mockup* (prototipo visual) de las prendas, sirviendo como guía gráfica estandarizada para el área de confección.
- **RF4:** El sistema debe permitir actualizar y visualizar el estado operativo de cada pedido (ej. Recepción, En Diseño, Sublimación, Confección, Entregado) para garantizar su trazabilidad en tiempo real.
- **RF5:** El sistema debe generar reportes filtrables de los pedidos pendientes por entregar, facilitando la planificación y monitoreo de la gerencia.

1.5.2. Requisitos no funcionales

- **RNF1: Usabilidad:** La interfaz debe ser intuitiva y fácil de usar para el personal del taller que no tiene formación técnica avanzada.

- **RNF2: Disponibilidad:** Al usar una base de datos en la nube (como Supabase), el sistema debe estar disponible para consulta en cualquier momento desde el taller o la oficina.
- **RNF3: Integridad:** El sistema debe garantizar que los datos de un pedido no se pierdan o se dupliquen accidentalmente.
- **RNF4: Seguridad:** El acceso debe estar restringido mediante usuarios y contraseñas según el cargo (Vendedor o Jefe de Taller).

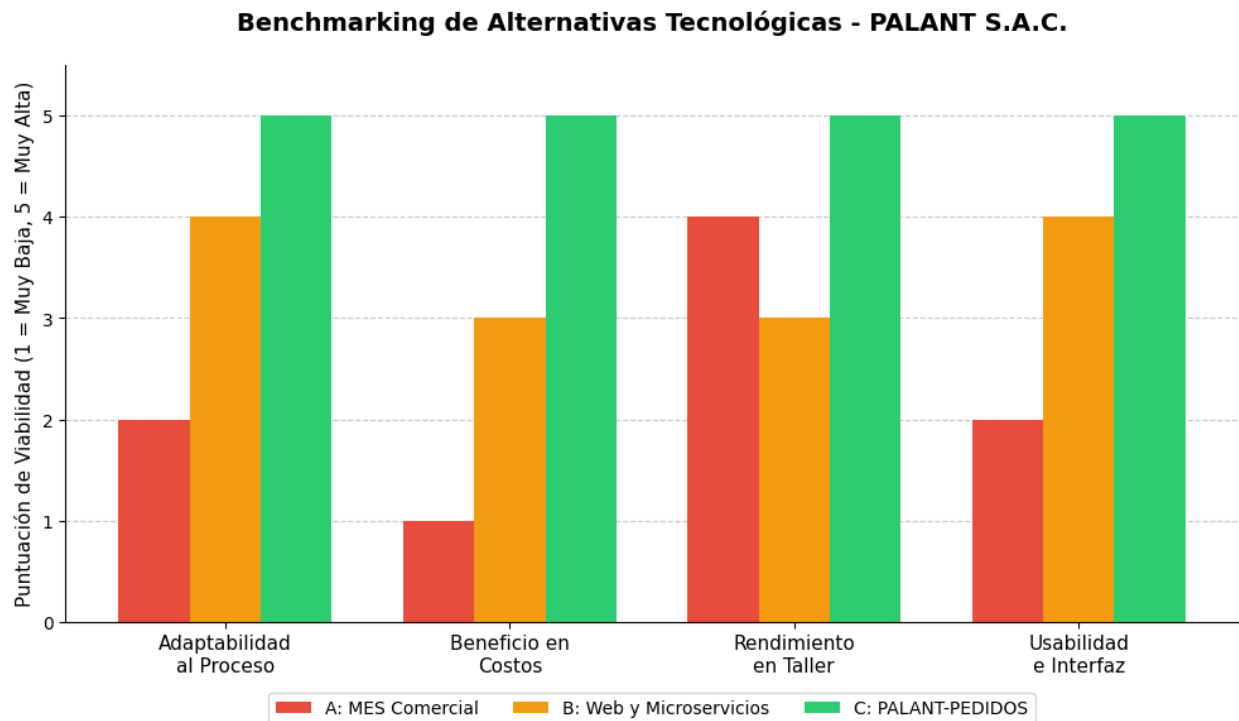
1.6. Modelo de negocio



Este Lean Canvas propone un software a medida (JavaFX/Supabase) para eliminar errores manuales y la falta de trazabilidad en Palant S.A.C. Su valor reside en optimizar la producción de uniformes para clubes de fútbol profesional, generando ingresos mediante el ahorro de insumos y eficiencia temporal.

1.7. Análisis comparativo de soluciones tecnológicas

Para determinar la viabilidad de la solución, se realizó un *benchmarking* evaluando tres alternativas frente a los requerimientos críticos de PALANT S.A.C.: la necesidad de personalización extrema (sublimación), el rendimiento en el entorno del taller, la usabilidad para operarios y el presupuesto de una PYME.



- **Selección Tecnológica** En base al análisis comparativo, se descartan las alternativas comerciales y web por representar un sobre costo y no adaptarse al nivel de usabilidad que requiere el personal operativo. Se selecciona la **Alternativa C** consolidando el siguiente *stack* tecnológico:
- **Frontend (Cliente):** Desarrollo en **JavaFX**. Se elige por su capacidad para crear interfaces gráficas de escritorio robustas, fluidas y personalizables, garantizando un rendimiento óptimo en las computadoras del taller sin las limitaciones de memoria de un navegador web.

- **Backend y Base de Datos:** Integración con **Supabase** (Backend-as-a-Service). Se selecciona como alternativa moderna a servidores físicos, ofreciendo una base de datos PostgreSQL en la nube con sincronización en tiempo real, lo que garantiza que los vendedores y el taller compartan la información sin latencia y con bajos costos de mantenimiento.

1.8. Planificación del proyecto

1.8.1. Metodología de desarrollo

Para la gestión y desarrollo del sistema PALANT-PEDIDOS, se ha seleccionado una metodología híbrida, la cual integra las mejores prácticas del rigor documental del PMBOK (Project Management Body of Knowledge) con la flexibilidad y adaptabilidad de un enfoque Ágil (basado en iteraciones y entregas continuas).

Esta elección se justifica de la siguiente manera:

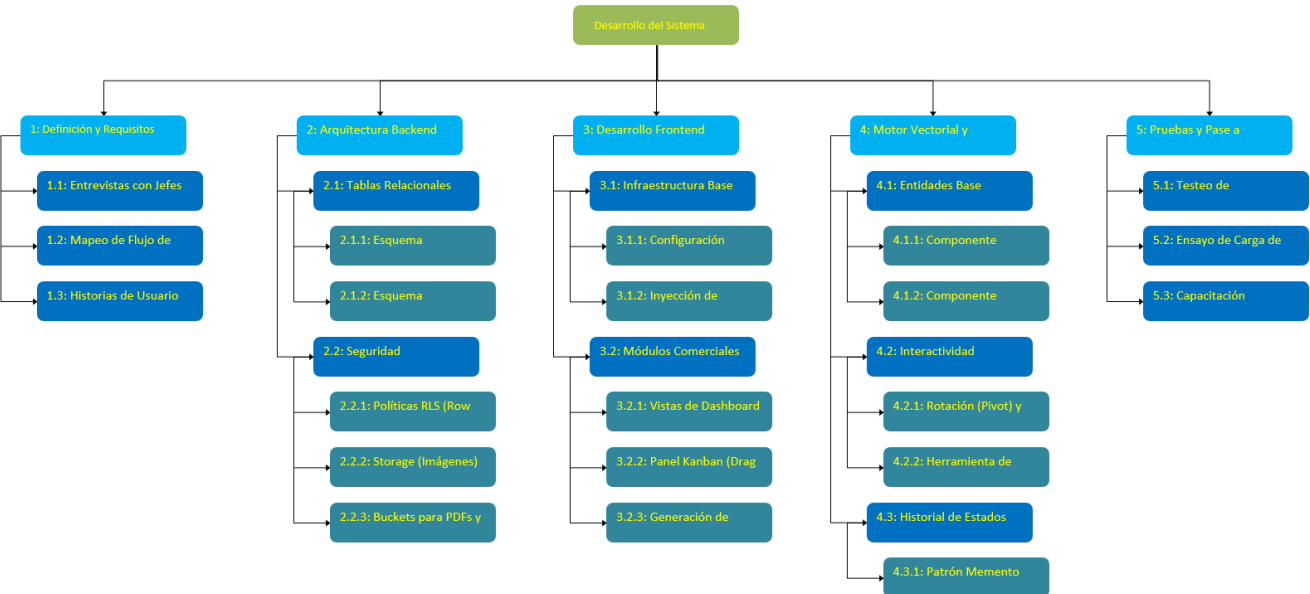
- **Componente PMBOK (Gestión y Control):** Se utiliza en las fases de inicio y planificación para definir claramente el alcance (lo que el sistema hará y lo que no), gestionar los riesgos, controlar el presupuesto y establecer el cronograma general del proyecto. Esto garantiza que el desarrollo tenga una base formal y estructurada, fundamental para el control administrativo de la empresa.
- **Componente Ágil (Ejecución y Desarrollo):** Dado que el sistema busca resolver un problema operativo visual en el taller, el desarrollo de las interfaces en JavaFX se trabajará de forma iterativa. Esto permite presentar prototipos funcionales de las pantallas al personal (diseñadores y costureros), obtener retroalimentación rápida y realizar ajustes sobre la marcha antes de la entrega final.

1.8.2. Project Charter

COMPONENTE	DETALLE TÉCNICO Y DE NEGOCIO
Nombre del Proyecto	Sistema Integrado de Control de Producción y Fichas Técnicas (PALANT-PEDIDOS).
Patrocinador	Gerencia General (Inversionista) y Jefatura de Taller (Usuario Principal) de PALANT S.A.C.
Director del Proyecto	Brando Kenny Caso Barzola
Duración Estimada	120 días calendario (Aprox. 4 meses de ejecución técnica).
Justificación del Negocio	El proceso de toma de pedidos deportivos tiene una tasa del 12% de error en transcripción de nombres/tallas, generando un sobrecosto de material. No existe conexión en tiempo real entre el área comercial (pedidos) y sublimación (taller).
Alcance y Stack Tecnológico	Frontend: Java, JavaFX 21, CSS, Maven. Motor Gráfico: Implementación matemática propia 2D (AffineTransform) para manipulación interactiva de textiles. Backend / DB: Supabase (PostgreSQL), Row Level Security (RLS), Supabase Storage.
Entregables Principales	1. Módulo Administrativo: CRUD de usuarios, roles, clientes y órdenes. 2. Módulo de Taller: Paneles Kanban de avance de producción. 3. Módulo de Diseño: Editor vectorial y de visualización de indumentaria.
Presupuesto Estimado	<i>Licencias Nube:</i> \$25/mes (Supabase Pro). <i>Hardware Taller:</i> Compra de 2 Tablets/Laptops industriales para planta. <i>Desarrollo Técnico:</i> [Valorizado en las horas de ingeniería].
Métricas de Éxito (KPIs)	- 99% de concordancia en tallas entregadas vs. pedidas. - < 2 segundos de tiempo de respuesta de carga de la ficha de diseño en taller. - 100% de los pedidos canalizados mediante la plataforma.
Riesgos Principales	1. Rechazo tecnológico por parte de operarios de planta (Mitigación: UI/UX simplificada e inductiva). 2. Caída del Wifi en el galpón de producción (Mitigación: Implementar modo lectura offline restringido).

Acta de Constitución del Proyecto (Project Charter). Resumen del alcance técnico, justificación del negocio y métricas de éxito (KPIs) establecidas

1.8.3. Estructura de desglose del trabajo (WBS)



El gráfico ilustra la Estructura de Desglose de Trabajo (WBS) del proyecto, organizando jerárquicamente el desarrollo en cinco grandes fases

1.8.4. Cronograma y diagrama de Gantt

			AbrilMayoJunioJulio																
N° Actividad	Inicio	Final	Semana1	Semana2	Semana3	Semana4	Semana5	Semana6	Semana7	Semana8	Semana9	Semana10	Semana11	Semana12	Semana13	Semana14	Semana15	Semana16	Semana17
Levantamiento de informacion	15/04/2026	25/04/2026																	
Aprovision de Project Charter	25/03/2026	30/04/2026																	
Diseño de arquitectura de BD	30/04/2026	10/05/2026																	
Diseño de interfaces(UI/UX)	10/05/2026	20/05/2026																	
Modulo de gestion de ordenes	20/05/2026	27/04/2026																	
Modulo interactivo	27/04/2026	17/05/2026																	
Conexion nube	17/05/2026	23/05/2026																	
Unitarias y funcionales	23/05/2026	02/06/2026																	
Piloto en taller PALANT	02/06/2026	10/06/2026																	
Capacitacion de personal	10/06/2026	18/06/2026																	
Paso a produccion y cierre	18/06/2026	05/07/2026																	

Diagrama de Gantt del proyecto ilustra la secuencia y duración estimada de las actividades clave para el desarrollo, pruebas y despliegue a producción del sistema PALANT-PEDIDOS.

1.8.5. Asignación de roles, hitos y entregables

Para garantizar la correcta ejecución del proyecto bajo el enfoque híbrido seleccionado, se definen los roles del equipo, el cronograma de hitos críticos y los entregables tangibles que se generarán a lo largo de las 17 semanas de desarrollo.

A. Asignación de Roles y Responsabilidades

- **Director y Desarrollador del Proyecto (Brando Kenny Caso Barzola):** Responsable de la gestión integral del proyecto, el levantamiento de información, el diseño de la arquitectura de la base de datos, el desarrollo del *frontend* y *backend*, así como de la ejecución de las pruebas y la capacitación del personal.
- **Patrocinador del Proyecto (Gerencia General de PALANT S.A.C.):** Responsable de proveer el presupuesto estimado (licencias nube y hardware para taller), validar los objetivos estratégicos y asegurar la viabilidad comercial del sistema.
- **Usuario Principal (Jefatura de Taller):** Responsable de validar los requisitos funcionales, probar las interfaces gráficas en situaciones reales de planta y facilitar la transición del personal de confección hacia la nueva plataforma.

B. Hitos del Proyecto

Los hitos representan los puntos de control clave donde se valida el avance del proyecto antes de pasar a la siguiente gran fase:

- **Hito 1 (Semana 2):** Formalización e inicio del proyecto mediante la aprobación del *Project Charter*.
- **Hito 2 (Semana 5):** Finalización del diseño de la arquitectura de la base de datos y esquemas relacionales.
- **Hito 3 (Semana 12):** Conclusión del desarrollo del núcleo del sistema (Módulos de Órdenes, Kanban e Interactivo).

- **Hito 4 (Semana 14):** Cierre de pruebas unitarias y finalización del piloto funcional en el taller de producción.
- **Hito 5 (Semana 17):** Culminación de la capacitación del personal, despliegue definitivo y cierre formal del proyecto.

C. Entregables Principales

- **Entregables de Gestión:** Acta de constitución (*Project Charter*), Estructura de Desglose de Trabajo (WBS) y cronograma de actividades en Diagrama de Gantt.
- **Entregables de Software:** Código fuente de la aplicación de escritorio **PALANT-PEDIDOS** (desarrollada en JavaFX 21 con Maven) y repositorio de base de datos PostgreSQL activo en la nube (**Supabase**).
- **Entregables de Soporte:** Manual de usuario digital e informe de control de calidad (pruebas unitarias y funcionales).

1.8.6. Herramientas de gestión utilizadas

La administración, seguimiento y control de las actividades del proyecto se apoyan en un conjunto de herramientas tecnológicas especializadas

- **Trello / Jira Software (Gestión de Tareas y Tablero Kanban):** Utilizado para mapear digitalmente la WBS del proyecto. Permite realizar el seguimiento diario de las tareas en desarrollo (*In Progress*), pendientes (*To Do*) y finalizadas (*Done*), asegurando la transparencia en el avance del código.
- **Lucidchart / Miro (Modelado y Diagramación):** Herramientas empleadas para el diseño visual del organigrama de la empresa, la Estructura de Desglose de Trabajo (EDT) y los diagramas de arquitectura de software previos a la fase de codificación.
- **Git y GitHub (Control de Versiones y Repositorio):** Plataforma fundamental para el almacenamiento seguro del código fuente del sistema. Permite mantener un historial

cronológico de los cambios, gestionar ramas de desarrollo y asegurar el respaldo del software ante contingencias técnicas.

- **Microsoft Excel (Planificación Temporal):** Herramienta utilizada para la construcción, cálculo de dependencias y actualización del Diagrama de Gantt del proyecto, permitiendo monitorear visualmente los desvíos en los tiempos estimados de cada actividad.

CAPÍTULO 2 – MARCO TEÓRICO

2.1. Fundamentos del desarrollo de software

En primer lugar, el desarrollo de software es una disciplina de la ingeniería informática enfocada en la aplicación de enfoques sistemáticos y cuantificables para transformar una necesidad operativa en una solución tecnológica funcional, garantizando que los procesos se ejecuten de manera eficiente y segura (Silva et al., 2026). De otro modo, para gestionar este proceso se utiliza el Ciclo de Vida del Desarrollo de Software (SDLC), un marco estructurado que abarca desde el análisis de requerimientos y el diseño de la arquitectura, hasta la implementación del código, las pruebas de calidad y su posterior despliegue. Asimismo, el propósito fundamental del SDLC es asegurar la entrega de un producto de alta calidad que cumpla con las restricciones del sistema y las expectativas del cliente dentro de los tiempos estimados, minimizando los riesgos inherentes a la construcción de plataformas tecnológicas (Rahman, 2024).

En segundo lugar, para ejecutar las fases del SDLC, la ingeniería de software se apoya en diversos paradigmas o modelos de desarrollo, tales como el modelo tradicional en Cascada (Waterfall), que exige completar cada etapa antes de avanzar, o el modelo Incremental, que divide el sistema en entregas funcionales parciales. Sin embargo, en el entorno actual destaca el enfoque Ágil (Agile), un modelo iterativo diseñado para adaptarse rápidamente a los cambios mediante ciclos cortos y comunicación directa con el usuario. A nivel nacional, la adopción de metodologías ágiles ha demostrado ser fundamental para cerrar brechas tecnológicas en los procesos operativos de las empresas peruanas, mitigando riesgos mediante entregas continuas y garantizando que el software final se ajuste a la realidad dinámica de los trabajadores (Zamora, 2023; Sarmiento Príncipe, 2024).

2.2. Análisis y diseño orientado a objetos

El Análisis y Diseño Orientado a Objetos (ADOO) es un enfoque de ingeniería que modela un sistema de software representando entidades del mundo real. Para estandarizar estos modelos, se utiliza el Lenguaje de Modelado Unificado (UML), el cual permite documentar tanto la estructura estática como el comportamiento dinámico del sistema antes de iniciar la fase de codificación.

A. Casos de uso

Son esquemas visuales que describen las interacciones funcionales entre los actores externos y el sistema, definiendo qué acciones puede realizar cada usuario para alcanzar un objetivo específico. A continuación, se presentan los casos de uso del sistema PALANT-PEDIDOS, detallando las interacciones entre los actores principales (Ventas, Diseño y Taller) y los módulos de gestión de órdenes de trabajo.

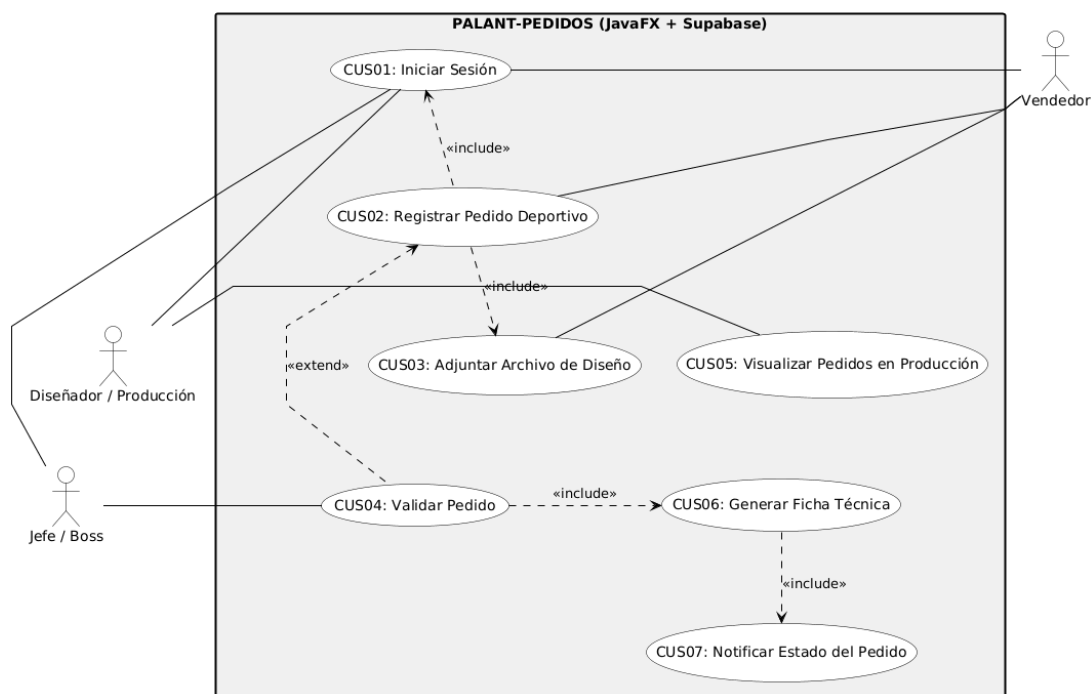


Diagrama de Casos de Uso. Ilustra los roles de los usuarios y sus permisos de interacción dentro del sistema de gestión.

B. Diagramas de clases

Representan el pilar estructural del modelado UML. Definen la arquitectura estática del software mostrando sus entidades, atributos, métodos y las relaciones (herencia, asociación) que existen entre ellas, sirviendo como base para el modelo relacional de datos. El siguiente diagrama ilustra la estructura central del sistema, definiendo entidades clave del negocio como Cliente, Usuario, DetallePedido, FichaTecnica, Pedido, fundamentales para la trazabilidad de la producción.

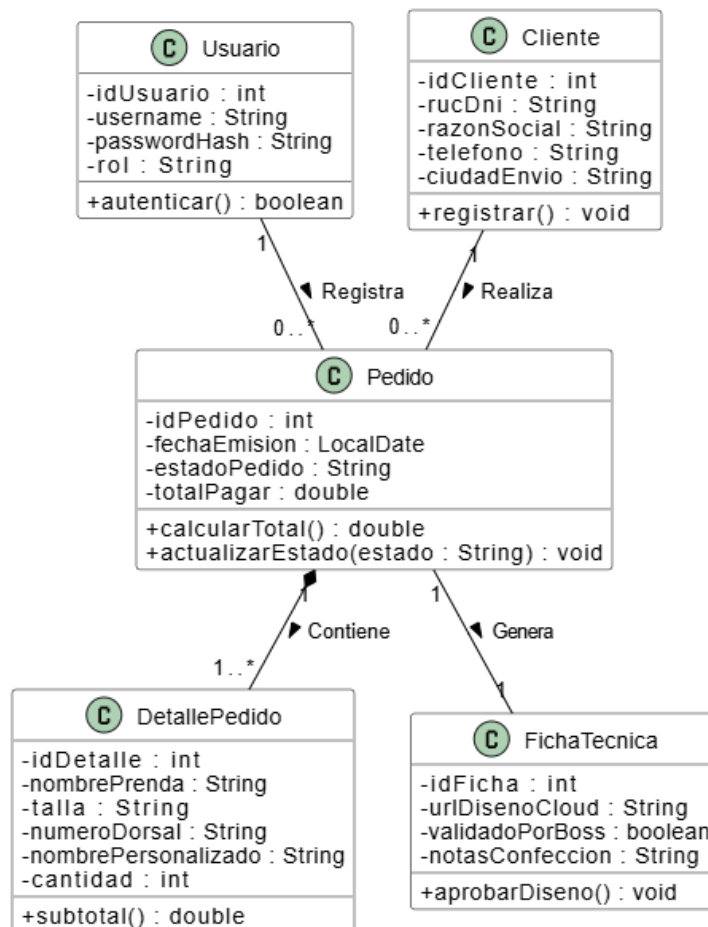


Diagrama de Clases. Estructura estática orientada a objetos que soporta la lógica de negocio y la base de datos del aplicativo.

C. Diagramas de secuencia

Pertenecen a la categoría de diagramas de comportamiento. Modelan el flujo lógico de un Caso de Uso específico, ilustrando cómo los objetos del sistema interactúan y se envían mensajes en un orden cronológico a lo largo del tiempo. Para comprender el flujo dinámico de la información, el diagrama modela el proceso de registro y actualización de estado de una orden, desde su ingreso inicial hasta que se visualiza en la interfaz del taller de confección.

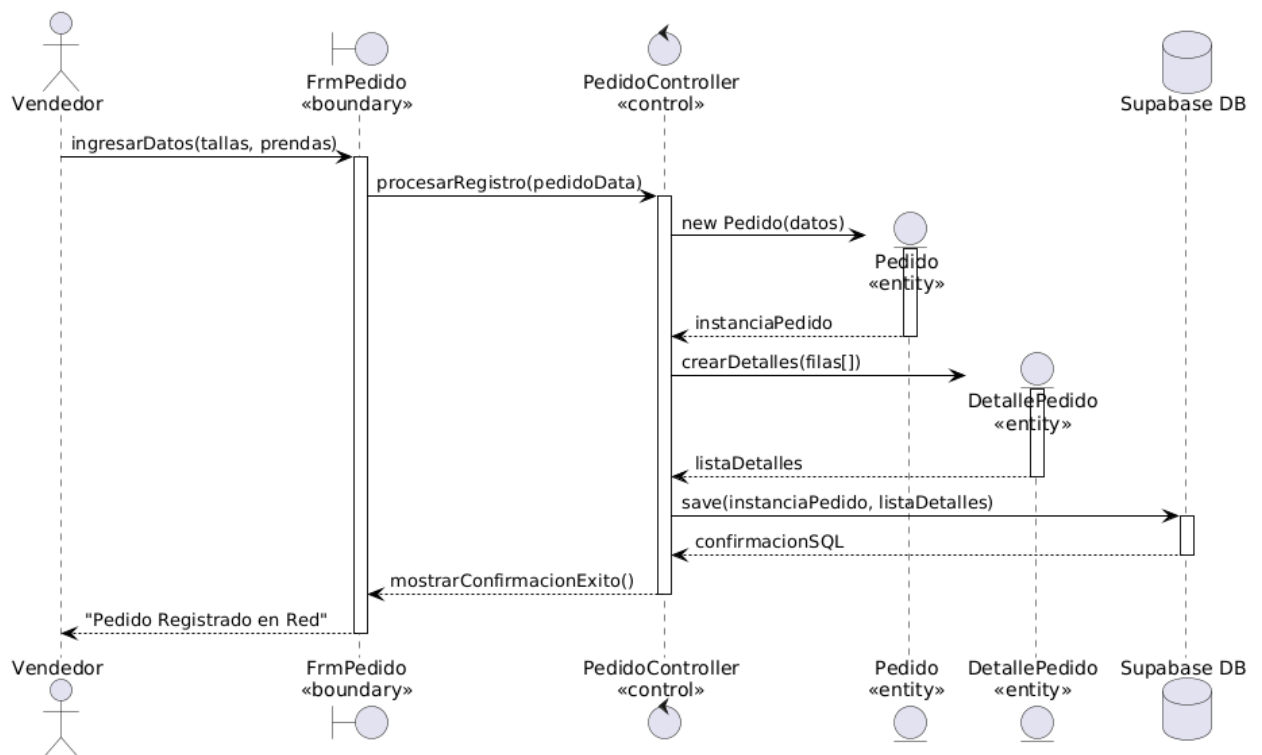


Diagrama de Secuencia. Flujo cronológico de mensajes entre los componentes del sistema durante el procesamiento de un pedido.

D. Diagramas de actividades

Son representaciones gráficas que muestran el flujo de control de una actividad a otra dentro del sistema, equivalentes a un diagrama de flujo pero adaptados al paradigma orientado a objetos. Son vitales para entender la lógica de los procesos paralelos. Finalmente, el siguiente diagrama detalla el flujo algorítmico y secuencial del ciclo de producción de la indumentaria deportiva, abarcando las transiciones desde el diseño y la sublimación hasta el ensamblado final.

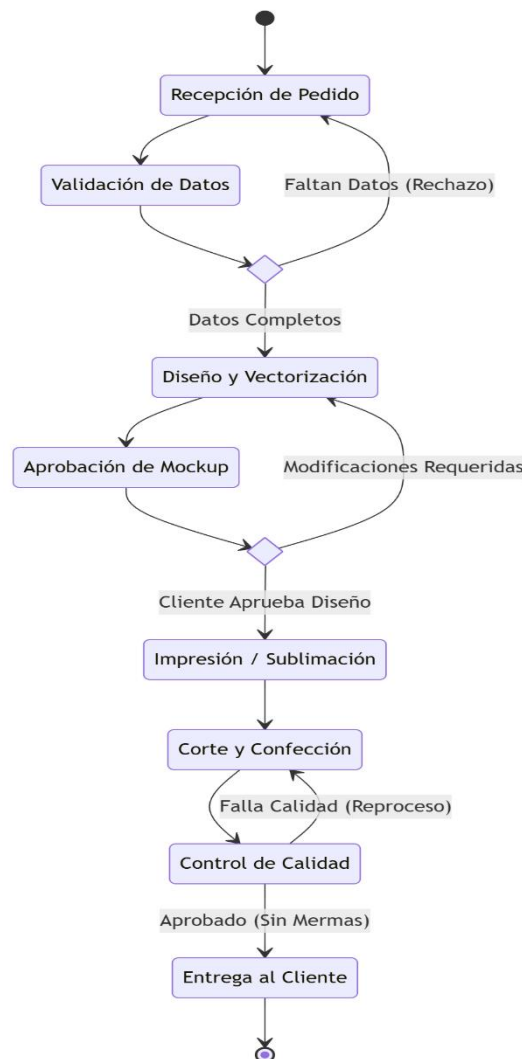


Diagrama de Actividades. Secuencia de pasos lógicos y decisiones dentro del flujo operativo del sistema.

2.3. Modelado de procesos de negocio (BPM)

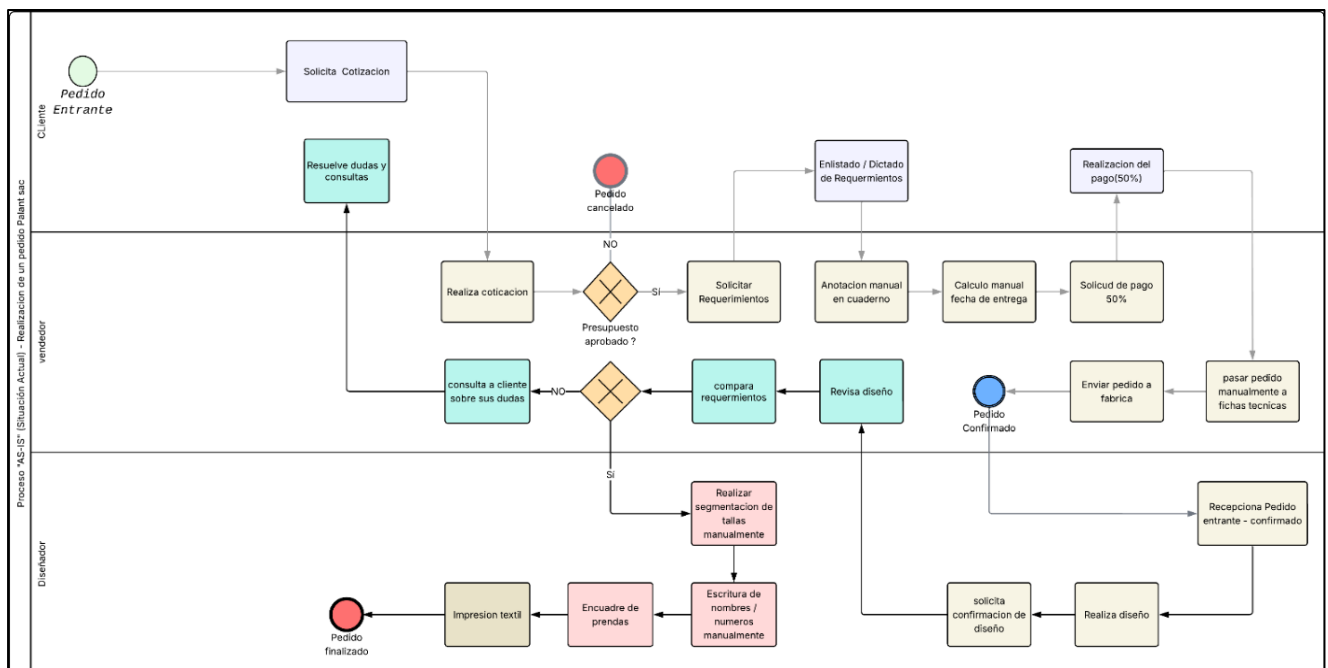
La Gestión de Procesos de Negocio (BPM) es una disciplina que combina metodologías de administración con tecnologías de la información. Su objetivo es identificar, diseñar y optimizar los flujos operativos de una organización para eliminar cuellos de botella y alinearlos con los objetivos estratégicos.

Introducción a BPMN (Business Process Model and Notation) BPMN es el estándar

internacional para el modelado gráfico de procesos de negocio. Proporciona una notación visual estructurada (mediante eventos, compuertas y carriles de responsabilidad) que resulta comprensible tanto para los analistas técnicos que desarrollan el sistema como para el personal operativo y la gerencia de la empresa.

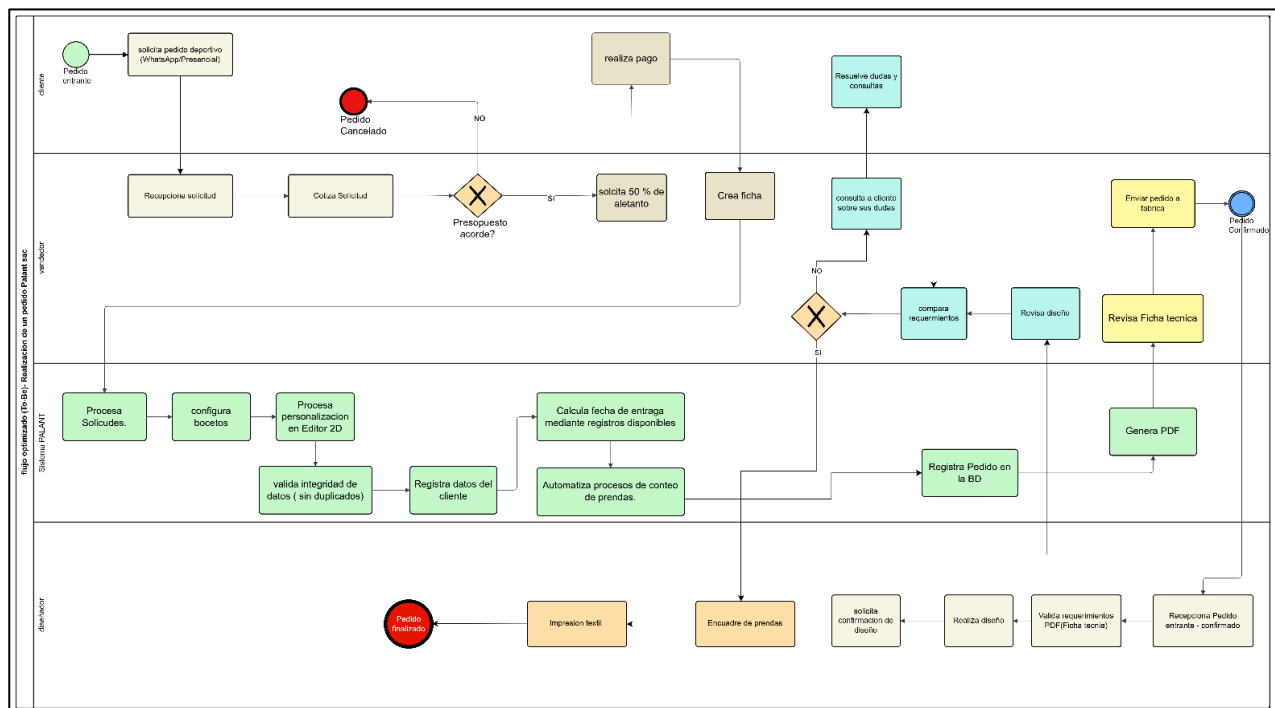
- **Diagramas del proceso actual (AS-IS)**

El modelo AS-IS (Tal como es) representa la fotografía actual de la ejecución de procesos antes de la intervención tecnológica. A continuación, se diagrama el flujo de trabajo manual de PALANT S.A.C., evidenciando la dependencia de hojas físicas y los retrasos por transcripción incorrecta de tallas entre el área comercial y el taller.



- **Diagramas del proceso propuesto (TO-BE)**

El modelo TO-BE (Como será) representa el escenario futuro optimizado. El siguiente diagrama ilustra cómo la implementación del sistema PALANT-PEDIDOS automatiza la comunicación, sincronizando en tiempo real las órdenes del área comercial directamente en los paneles digitales del taller.

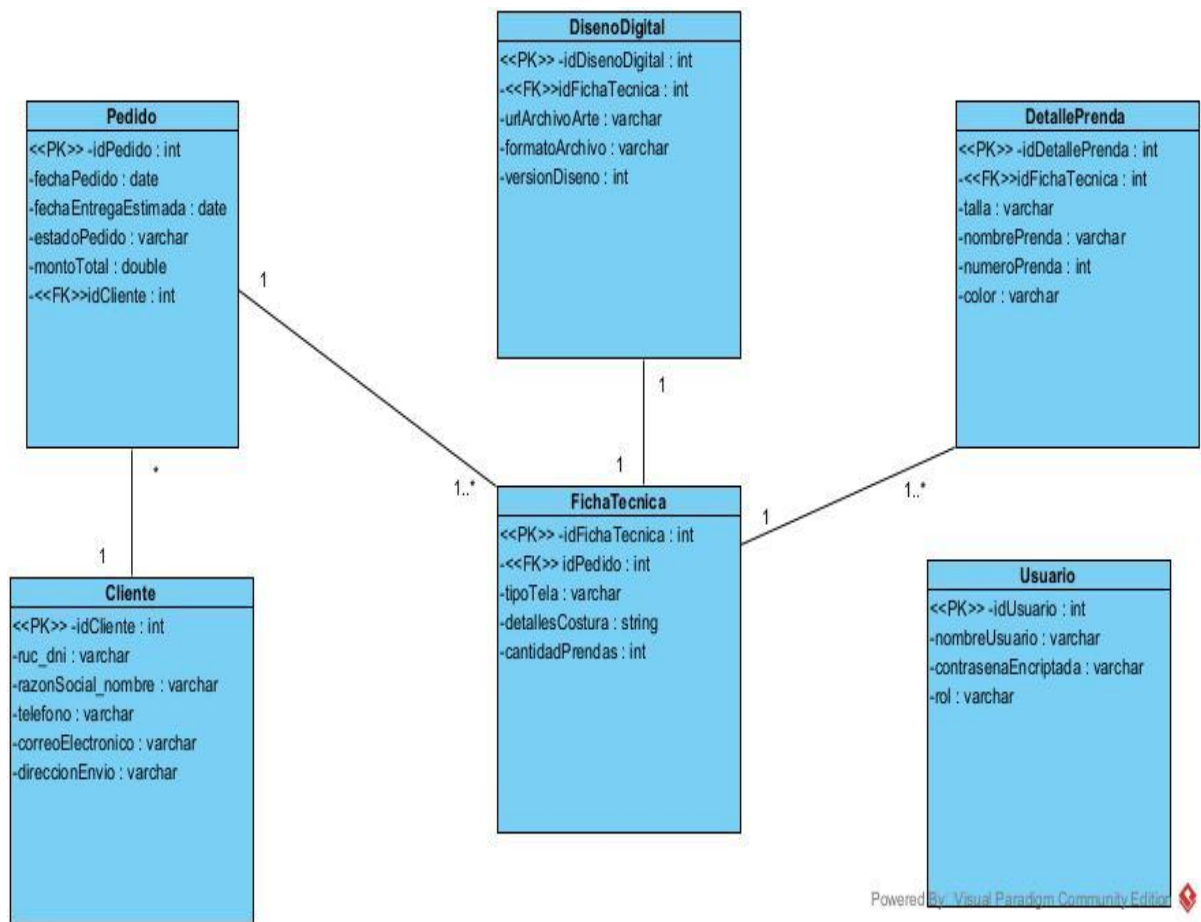


2.4. Base de datos

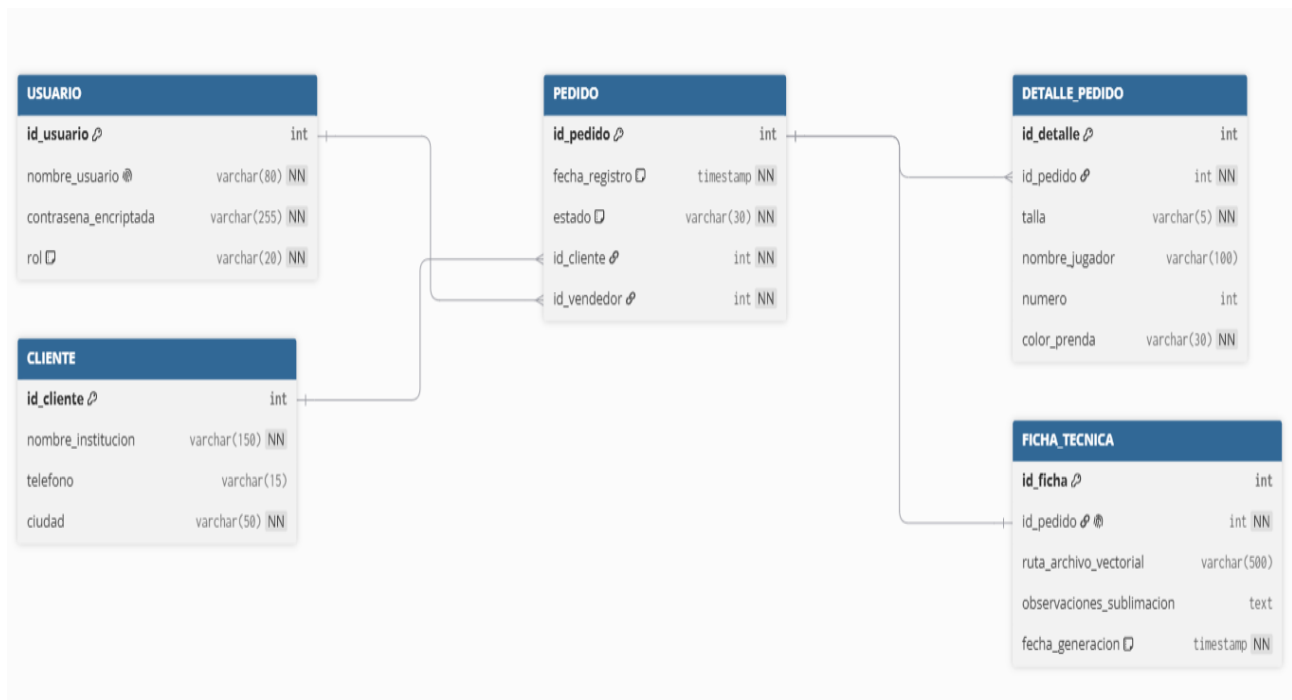
Una base de datos es un conjunto de información organizada sistemáticamente para responder a los requerimientos operativos de una empresa. En ingeniería de software, su diseño requiere una arquitectura estructurada en tres capas para garantizar la integridad, evitar la redundancia y asegurar la escalabilidad del sistema a largo plazo.

- **Modelo conceptual** Es la representación de más alto nivel. Su objetivo es identificar las entidades clave del negocio y las relaciones entre ellas, sin enfocarse en los detalles técnicos ni en el software que se utilizará.

A continuación, se presenta el modelo conceptual de PALANT-PEDIDOS, identificando las entidades principales del proceso productivo (Cliente, Pedido, Prenda, Diseño y Usuario) y cómo interactúan en el flujo de trabajo.



Modelo Conceptual de la base de datos. Identificación de entidades y relaciones a nivel de negocio.



- **Modelo lógico** Funciona como el paso intermedio y traduce el modelo conceptual a estructuras detalladas. Aquí se definen los atributos de cada entidad y se establecen las claves primarias (PK) y foráneas (FK) para conectar la información mediante reglas de cardinalidad. El siguiente diagrama detalla la estructura lógica del sistema, estableciendo las dependencias que garantizan que una orden de trabajo (Pedido) centralice las prendas solicitadas y los diseños adjuntos

Modelo Lógico de datos. Definición de atributos, claves primarias (PK), foráneas (FK) y cardinalidad entre las entidades.

- **Modelo físico** Es la implementación real del modelo lógico en un Sistema Gestor de Bases de Datos (SGBD) específico. En esta capa se definen los tipos de datos exactos, las restricciones de campos y las reglas de almacenamiento.
- Para el presente proyecto, el modelo físico está diseñado para ser desplegado en **PostgreSQL mediante la plataforma Supabase**. El diagrama refleja los tipos de datos nativos de SQL (UUID, VARCHAR, TIMESTAMP, NUMERIC) y las restricciones de obligatoriedad (NOT NULL, UNIQUE) necesarias para el funcionamiento del *backend*.

```

-- 2. Creación de la tabla USUARIO (debe existir antes que PEDIDO)

CREATE TABLE usuario (

    id_usuario    SERIAL PRIMARY KEY,

    nombre_usuario VARCHAR(80) NOT NULL UNIQUE,

    contrasena_encryptada VARCHAR(255) NOT NULL,

    rol           VARCHAR(20) NOT NULL

                CHECK (rol IN ('Vendedor','Diseñador','Jefe'))

);

-- 3. Creación de la tabla PEDIDO

CREATE TABLE pedido (

    id_pedido     SERIAL PRIMARY KEY,

    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    estado        VARCHAR(30) NOT NULL DEFAULT 'Pendiente',

    id_cliente    INT NOT NULL,

    id_vendedor   INT NOT NULL,

    CONSTRAINT fk_pedido_cliente

        FOREIGN KEY (id_cliente)

        REFERENCES cliente(id_cliente) ON UPDATE CASCADE,

    CONSTRAINT fk_pedido_vendedor

        FOREIGN KEY (id_vendedor)

        REFERENCES usuario(id_usuario) ON UPDATE CASCADE

        REFERENCES pedido(id_pedido) ON DELETE CASCADE

);

```

Modelo Físico de datos. Esquema de tablas diseñado con tipos de datos de PostgreSQL para la implementación en la nube mediante Supabase.

```
-- =====

-- SCRIPT DDL - SISTEMA PALANT-PEDIDOS

-- Base de Datos: PostgreSQL via Supabase

-- =====

-- 1. Creación de la tabla CLIENTE

CREATE TABLE cliente (

    id_cliente    SERIAL PRIMARY KEY,

    nombre_institucion VARCHAR(150) NOT NULL,

    telefono      VARCHAR(15),

    ciudad        VARCHAR(50)  NOT NULL

);
```

```
);-- 4. Creación de la tabla DETALLE_PEDIDO

CREATE TABLE detalle_pedido (

    id_detalle    SERIAL PRIMARY KEY,

    id_pedido     INT NOT NULL,

    talla         VARCHAR(5)  NOT NULL,

    nombre_jugador VARCHAR(100),

    numero        INT,

    color_prenda  VARCHAR(30) NOT NULL,

    CONSTRAINT fk_detalle_pedido

        FOREIGN KEY (id_pedido));

-- 5. Creación de la tabla FICHA_TECNICA

CREATE TABLE ficha_tecnica (

    id_ficha      SERIAL PRIMARY KEY,

    id_pedido     INT NOT NULL UNIQUE,

    ruta_archivo_vectorial VARCHAR(500),

    observaciones_sublimacion TEXT,

    fecha_generacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_ficha_pedido

        FOREIGN KEY (id_pedido)

        REFERENCES pedido(id_pedido) ON DELETE CASCADE
```

2.5. Lenguajes de programación y herramientas

El desarrollo de soluciones de software a medida requiere la selección estratégica de una pila tecnológica (*stack*) que garantice la mantenibilidad, el rendimiento y la seguridad de la aplicación a lo largo de todo su ciclo de vida.

Tecnologías utilizadas

- **Java:** Es un lenguaje de programación de propósito general, fuertemente tipado y orientado a objetos. Su robustez, manejo automático de memoria (Garbage Collector) y su filosofía multiplataforma lo convierten en el estándar de la industria para el desarrollo de sistemas empresariales seguros y concurrentes (Oracle, 2023). En este proyecto, actúa como el motor central de la lógica de negocio.
- **PostgreSQL (Backend-as-a-Service):** Es un potente motor de base de datos relacional de código abierto. Para este sistema, se implementa bajo el modelo BaaS (Backend como Servicio) a través de la plataforma en la nube Supabase. Esto permite gestionar autenticaciones, políticas de seguridad a nivel de fila (RLS) y sincronización de datos en tiempo real entre el área comercial y el taller productivo sin la necesidad de administrar servidores físicos locales.

Frameworks y librerías

Para agilizar el desarrollo y evitar programar funcionalidades base desde cero, el sistema se apoya en los siguientes *frameworks* y librerías especializadas:

- **JavaFX:** Es un *framework* nativo de la plataforma Java diseñado para la creación de aplicaciones cliente enriquecidas (Rich Client Applications). Permite construir interfaces gráficas de usuario (GUI) modernas y fluidas, separando la lógica del programa del diseño visual mediante el uso de hojas de estilo (CSS) y archivos de marcado (FXML).
- **Apache Maven:** Es una herramienta de gestión y comprensión de proyectos de software. Automatiza el proceso de compilación del código y administra de forma

centralizada las dependencias (librerías externas) a través del archivo de configuración POM (Project Object Model).

- **Librerías de soporte técnico:** Se integran herramientas clave como **Google Guava** para optimizar el manejo de colecciones y estructuras de datos complejas; **Apache POI** para la generación e interoperabilidad de reportes documentales; y **Logback** para el registro estructurado de eventos del sistema (logs), lo cual es vital para el monitoreo y mantenimiento técnico.

Herramientas de control de versiones (Git)

Git es un sistema de control de versiones distribuido, de código abierto, diseñado para registrar y administrar con alta eficiencia el historial de cambios en el código fuente de un proyecto. A diferencia de los modelos centralizados, Git permite la creación de ramas (*branches*) paralelas, facilitando que el desarrollador trabaje en nuevas funcionalidades (como el panel Kanban o el motor vectorial) de forma aislada sin comprometer la versión estable del sistema principal (Instituto Nacional de Tecnologías Educativas [INTEF], 2024). Para el almacenamiento seguro y remoto de estos repositorios, se hace uso de la plataforma **GitHub**.

CAPÍTULO 3 – DISEÑO DEL SISTEMA

3.1. Diseño de interfaz de usuario (UX/UI)

El diseño de la interfaz de usuario (UI) y la experiencia de usuario (UX) es la fase de la ingeniería de software encargada de estructurar los entornos visuales con los que interactúan los operadores. Su objetivo principal es garantizar que el sistema sea intuitivo, accesible y eficiente, reduciendo al mínimo la curva de aprendizaje del personal de la empresa.

A. Principios de diseño de interfaz

Para el desarrollo del sistema **PALANT-PEDIDOS**, el diseño de las pantallas en JavaFX se rige bajo los principios fundamentales de la usabilidad y la heurística de Nielsen:

- **Consistencia y Estándares:** Toda la aplicación utiliza una paleta cromática homogénea, tipografías estandarizadas y botones con funciones predecibles. Esto asegura que el usuario no tenga que preguntarse si diferentes palabras o acciones significan lo mismo en distintas pantallas.
- **Visibilidad del Estado del Sistema:** El sistema siempre mantiene informado al usuario de lo que está ocurriendo en el *backend* a través de alertas visuales en tiempo real (por ejemplo, notificaciones de éxito al guardar una orden en Supabase o barras de carga al procesar una imagen).
- **Flexibilidad y Eficiencia de Uso:** La interfaz está optimizada tanto para usuarios novatos (con menús guiados e intuitivos) como para usuarios expertos, mediante aceleradores de teclado y el uso de un panel Kanban con tecnología de arrastrar y soltar (*drag-and-drop*) para agilizar el cambio de estados en el taller.
- **Prevención de Errores y Mitigación del Rechazo Tecnológico:** Sabiendo que los operarios en planta pueden ser reacios a sistemas complejos, el diseño UI se ha

simplificado al máximo. Los campos de texto críticos (como el ingreso de tallas y nombres) cuentan con validaciones automáticas y menús desplegables para evitar el error humano de transcripción en el galpón de producción.

B. Prototipado y navegación

La arquitectura de información y el flujo de navegación del sistema han sido validados previamente mediante prototipos de interfaz, asegurando una transición fluida entre los diferentes módulos del aplicativo. La navegación es de tipo **centralizada y basada en roles**, lo que significa que tras pasar por la pantalla de inicio de sesión, el sistema despliega un menú lateral dinámico que expone únicamente los módulos autorizados para el usuario activo:

- **Módulo Administrativo / Ventas:** Acceso al formulario de registro de órdenes de trabajo, control de clientes y visualización de la lista general de pedidos.
- **Módulo de Taller:** Vista directa al tablero Kanban de avance de producción, optimizado para el uso ágil en las estaciones de trabajo de planta.
- **Módulo de Diseño:** Editor y visualizador interactivo de indumentaria textil para la carga, previsualización y aprobación de mockups digitales.

A continuación, se presentan las capturas de las interfaces o diagramas de navegación diseñados para el sistema

- **Pantalla de Inicio**



1- Parte de Diseño y técnica

Archivo Guardar

PALANT
SISTEMA DE PEDIDOS

ENVÍO Disño CÓDIGO PRIORIDAD ENTREGA

Completar datos Jugador P26-M14 Normal VIE 22/05/2026 CONTINUAR

1. Diseño y Técnica 2. Logos / Sponsors 3. Lista de Jugadores 4. Ficha Técnica

Configuración

INICIAR PEDIDO
Elige una opción antes de seleccionar la prenda.

Nuevo Pedido
Abrir Proyecto

Seleccione "Nuevo Pedido" o "Abrir Proyecto" para comenzar.

PALANT
SISTEMA DE PEDIDOS

ENVÍO Disño CÓDIGO PRIORIDAD ENTREGA

Completar datos Jugador P26-M14 Normal VIE 22/05/2026 CONTINUAR

1. Diseño y Técnica 2. Logos / Sponsors 3. Lista de Jugadores 4. Ficha Técnica

Configuración

Camiseta Cambiar

GÉNERO (TORSO)

Hombre Mujer

TIPO DE TELA

WILL

TIPO DE CORTE (MANGA)

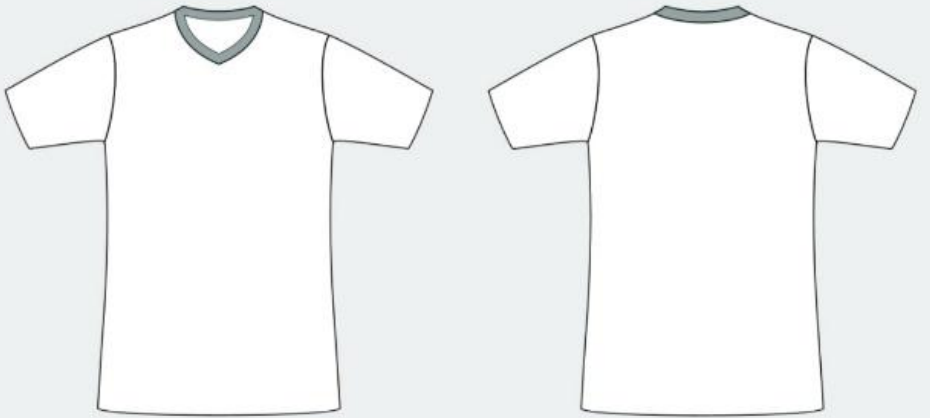
Cuadrado Redondo

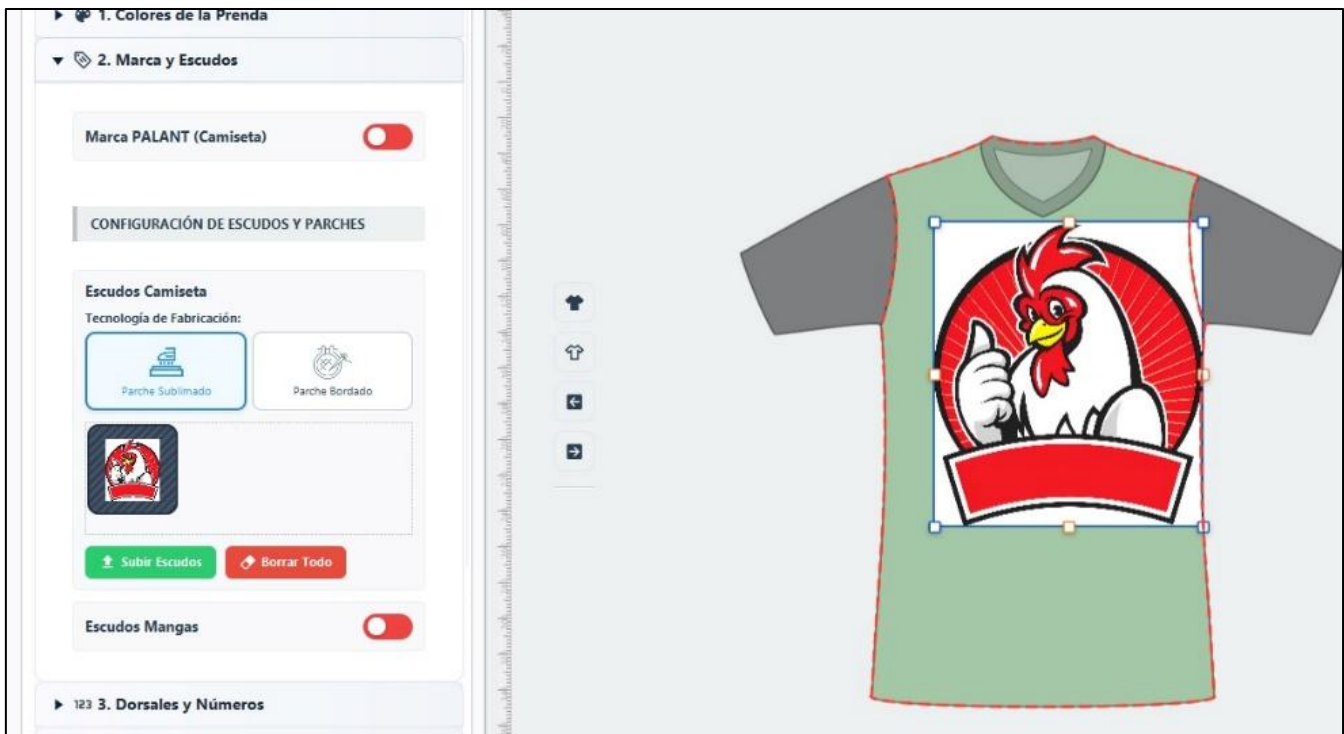
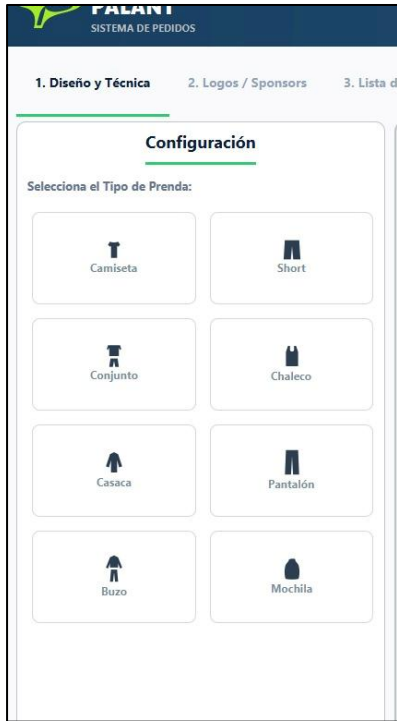
LARGO DE MANGA

Catálogo de Mangas
Selección actual: Manga Corta

TIPO DE CUELLO

Catálogo de Cuellos





Prototipo de la interfaz principal de PALANT-PEDIDOS. Se observa la distribución del panel lateral de navegación y el área de trabajo centralizada.

2- Lista de Jugadores

Archivo (sin guardar) * x

PALANT
SISTEMA DE PEDIDOS

ENVÍO DISEÑO CÓDIGO PRIORIDAD ENTREGA
 Completar datos Jugador P26-M14 Normal VIE 22/05/2026 CONTINUAR

1. Diseño y Técnica 2. Logos / Sponsors 3. Lista de Jugadores 4. Ficha Técnica

Masiva / Correlativa Especifica Simples Manual

1. Configuración
 Género Base: HOMBRE Número Inicial: 2
 Numeración: ☒ Global Continua ☐ Por Talla

2. Acciones
 Agregar Bloque de Talla

Distribución de Tallas
 Talla: M Cantidad: 12 GENERAR LISTA

#	Nombre	Talla	Número	Género	Manga	Short	Tipo Inf.	T. Short
1	JUGADOR	M	2	HOMBRE	CORTA	☐		☐
2	JUGADOR	M	3	HOMBRE	CORTA	☐		☐
3	JUGADOR	M	4	HOMBRE	CORTA	☐		☐
4	JUGADOR	M	5	HOMBRE	CORTA	☐		☐
5	JUGADOR	M	6	HOMBRE	CORTA	☐		☐
6	JUGADOR	M	7	HOMBRE	CORTA	☐		☐
7	JUGADOR	M	8	HOMBRE	CORTA	☐		☐
8	JUGADOR	M	9	HOMBRE	CORTA	☐		☐
9	JUGADOR	M	10	HOMBRE	CORTA	☐		☐
10	JUGADOR	M	11	HOMBRE	CORTA	☐		☐
11	JUGADOR	M	12	HOMBRE	CORTA	☐		☐
12	JUGADOR	M	13	HOMBRE	CORTA	☐		☐

Borrar Nombres Borrar Números VACIAR TODO

3- Ficha Técnica

Archivo (sin guardar) * x

PALANT
SISTEMA DE PEDIDOS

ENVÍO DISEÑO CÓDIGO PRIORIDAD ENTREGA
 Completar datos Jugador P26-M14 Normal VIE 22/05/2026 IMPRIMIR / PDF

1. Diseño y Técnica 2. Logos / Sponsors 3. Lista de Jugadores 4. Ficha Técnica

DESOLUC DE PRENDAS - PULS HOMBRE

Prenda / Talla	M	TOTAL
Camiseta (m)	12	12



CAMISETA

Carac.	QUOTIPISO
Cuello	V
Tubo	NO
Manga	CORTA

LOGOS



PECHO (FRONTAL)
 MARCA DE REGISTRO

PALETA DE COLORES
☒ Verde ☐ Rojo ☐ Negro

FICHA TÉCNICA

PROYECTO: 2024-2025
 DISEÑO: JUAN CARLOS ANDRÉS
 CANTIDAD: 12

LISTA DE JUGADORES

SOLO CAMISETA - CORTA MANGA - MANGA CORTA (NUMERACIÓN)		
TALLA	CANTIDAD	NOMBRE
M	12	1-12-13-14-15-16-17-18-19-20-21-22-23-24-25-26-27-28-29-30-31-32-33-34-35-36-37-38-39-40-41-42-43-44-45-46-47-48-49-50-51-52-53-54-55-56-57-58-59-60-61-62-63-64-65-66-67-68-69-70-71-72-73-74-75-76-77-78-79-80-81-82-83-84-85-86-87-88-89-90-91-92-93-94-95-96-97-98-99-100

60%

A. Mapeo de reportes clave

Una de las funciones esenciales de la plataforma es la transformación de los datos operativos en información útil para la toma de decisiones estratégicas y el control de calidad en el taller. El sistema mapea y genera de forma automatizada los siguientes reportes clave:

- **Ficha Técnica Digital de Producción:** Reporte en formato digital (PDF) generado por el sistema que consolida de manera idéntica los nombres, números y tallas de cada prenda solicitada en una orden, enlazándola directamente con el archivo vectorizado del diseño. Este reporte elimina la necesidad de transcribir datos en hojas impresas.
- **Reporte de Rendimiento e Indicadores (KPIs):** Panel estadístico que extrae datos históricos de la base de datos Supabase para medir el porcentaje de concordancia en tallas entregadas y calcular el tiempo promedio que toma una orden en pasar desde la etapa de diseño hasta la confección final.

3.2. Documentación técnica

La documentación técnica es el conjunto de manuales, guías y registros estructurados que describen tanto la arquitectura interna del software como su modo de operación. Su objetivo es garantizar que el sistema sea mantenible en el tiempo por otros programadores y que pueda ser operado correctamente por los usuarios finales sin necesidad de asistencia técnica constante.

A. Manual de desarrolladores (Javadoc, Markdown)

El manual del desarrollador está diseñado exclusivamente para el equipo técnico o futuros ingenieros que necesiten realizar tareas de mantenimiento, actualización o escalabilidad en el código fuente de la plataforma. Para este proyecto, la documentación técnica se divide en dos formatos estándar:

- **Javadoc:** Se utiliza para la documentación interna del código fuente en Java. Mediante comentarios estructurados en las cabeceras de cada clase y método, Javadoc genera automáticamente archivos HTML que describen la API del sistema. En **PALANT-PEDIDOS**, esto es fundamental para documentar de forma limpia las clases del motor gráfico 2D, los controladores de las interfaces JavaFX y los servicios de conexión que interactúan con la base de datos en Supabase.
- **Markdown:** Es un lenguaje de marcado ligero que se utiliza para redactar el archivo principal de bienvenida (README.md) en el repositorio de GitHub. Este componente funciona como la guía de despliegue inicial, detallando los requisitos del entorno (versión de Java JDK 21, configuración de Maven) y los pasos exactos que un programador debe seguir para clonar, compilar y ejecutar el proyecto por primera vez

B. Manual de usuario

El manual de usuario es un documento no técnico orientado a los trabajadores de la empresa que interactúan directamente con el aplicativo en sus labores diarias. Funciona como una guía interactiva paso a paso, apoyada en capturas de pantalla, para asegurar que los flujos de trabajo se ejecuten sin errores. Este manual se estructura bajo los roles operativos del sistema:

- **Guía para el Área Comercial (Ventas):** Explica visualmente cómo iniciar sesión, cómo registrar un cliente nuevo y cuáles son los pasos obligatorios para ingresar un pedido al sistema, incluyendo la selección de tallas, nombres, números y la carga del archivo de diseño técnico.

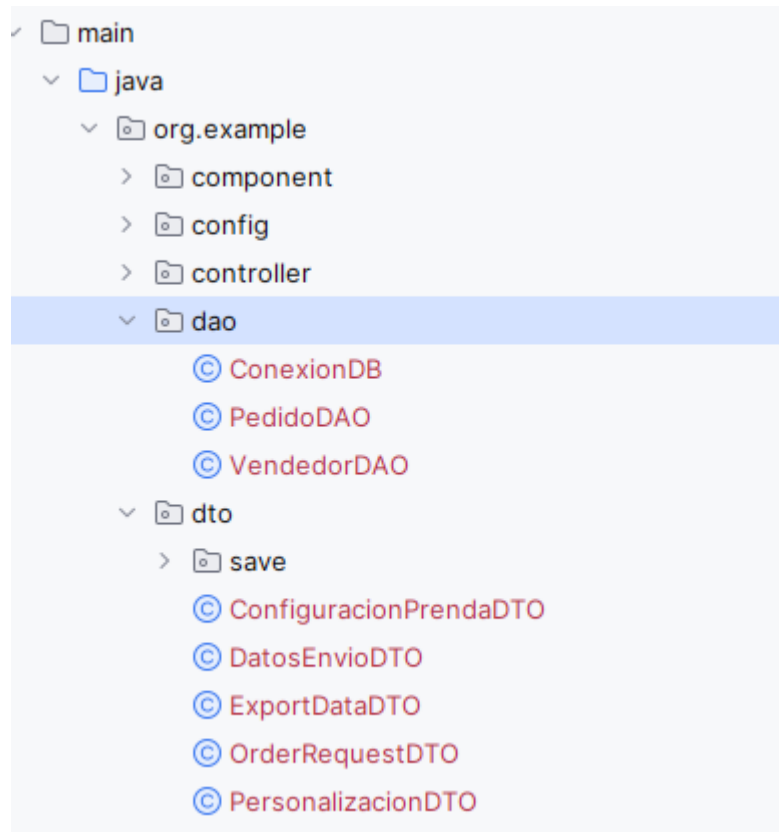
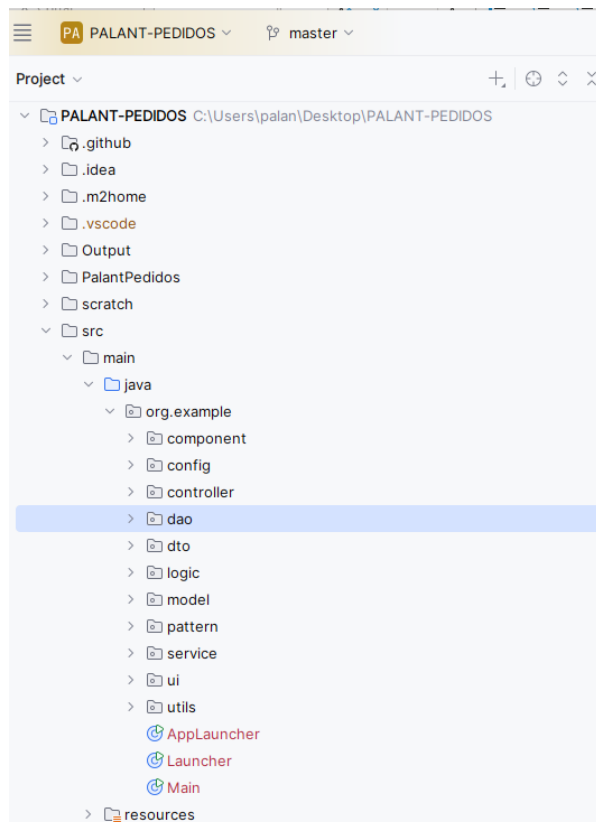
CAPÍTULO 4 – DESARROLLO DE LA SOLUCIÓN

4.1. Arquitectura del sistema

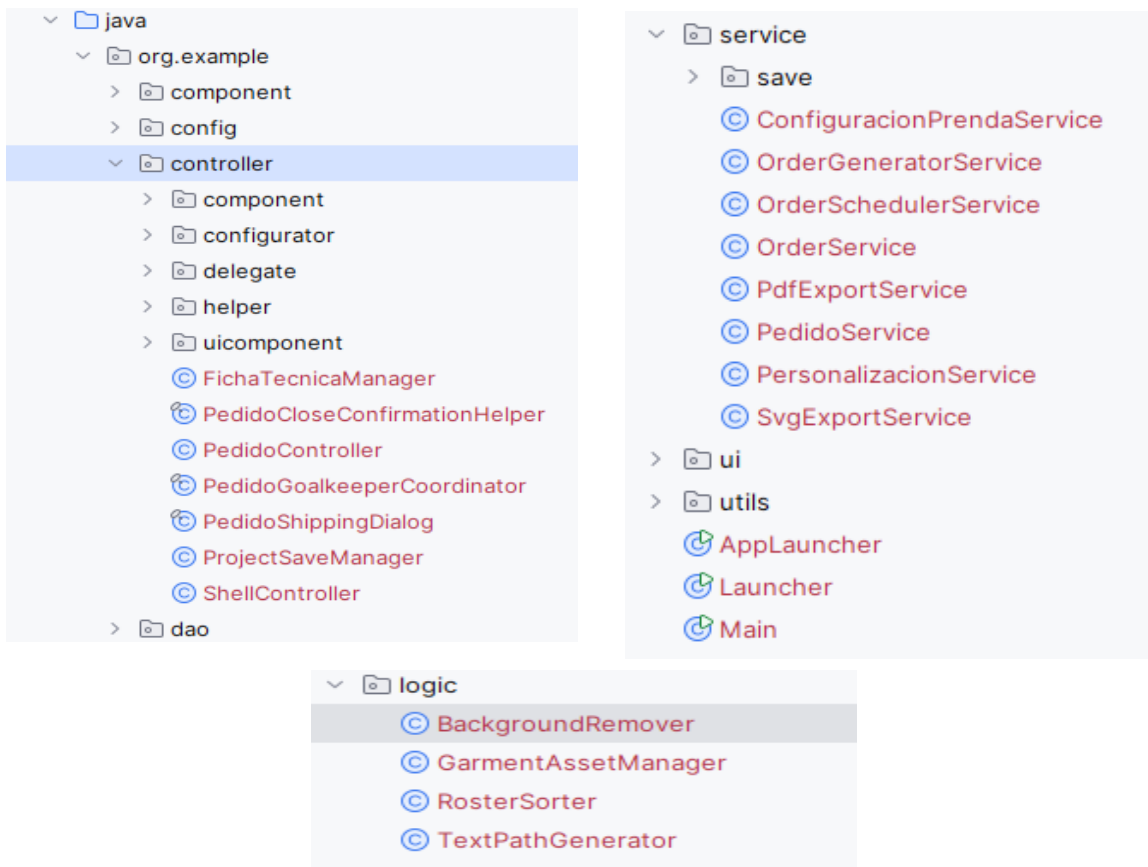
A. Patrones utilizados (MVC, DAO, SOLID)

El sistema PALANT-PEDIDOS ha sido construido sobre una arquitectura robusta y escalable que aplica varios patrones de diseño de software. En primer lugar, se utilizó el patrón MVC (Modelo-Vista-Controlador) para estructurar el desarrollo. Las vistas se implementaron mediante archivos FXML en JavaFX para las interfaces de usuario, separadas de los controladores (controller) que manejan las interacciones y los modelos (model) que encapsulan los datos del negocio.

Adicionalmente, se empleó el patrón DAO (Data Access Object) en el paquete dao para abstraer y encapsular todos los accesos a la base de datos en la nube (Supabase). Esto permite que la lógica del negocio no dependa de sentencias SQL directas, facilitando el mantenimiento.

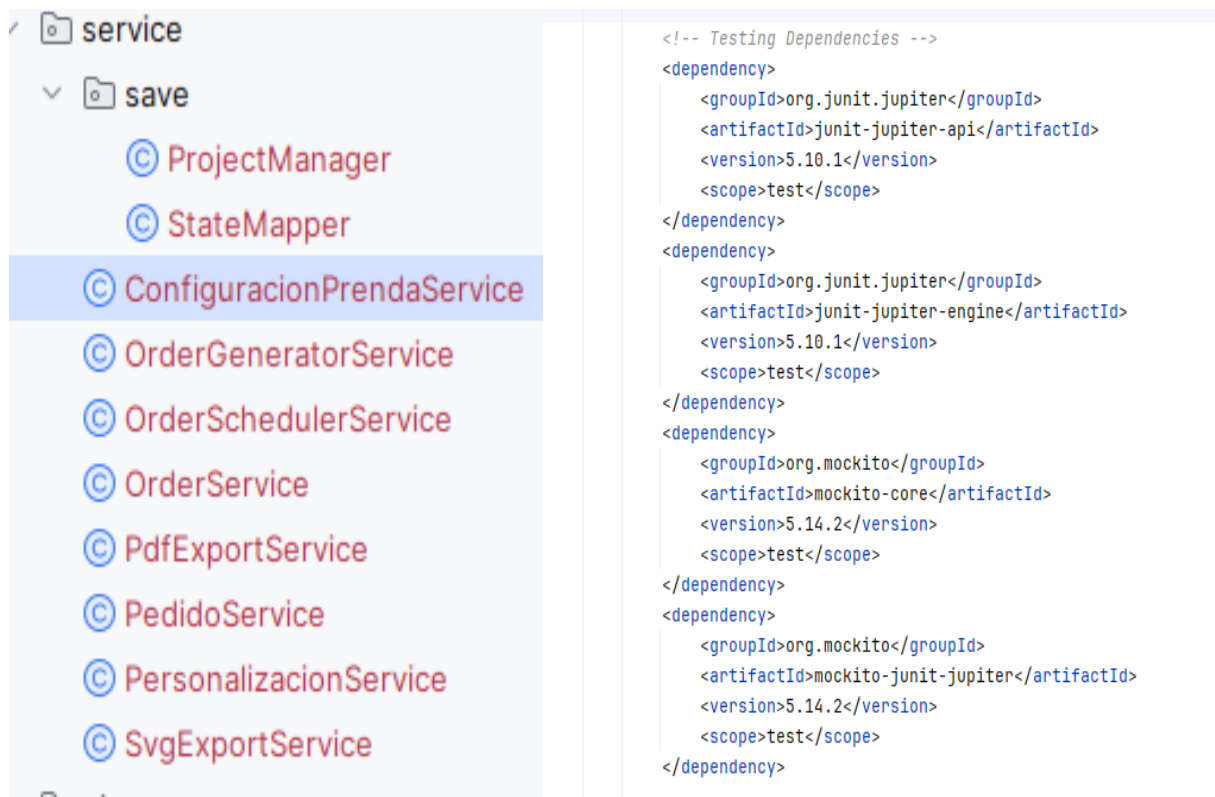


Finalmente, toda la programación orientada a objetos en los paquetes de servicios (service y logic) siguió estrictamente los Principios SOLID, garantizando una alta cohesión, bajo acoplamiento y que cada clase tenga una única responsabilidad (Single Responsibility Principle).



B. Enfoque TDD

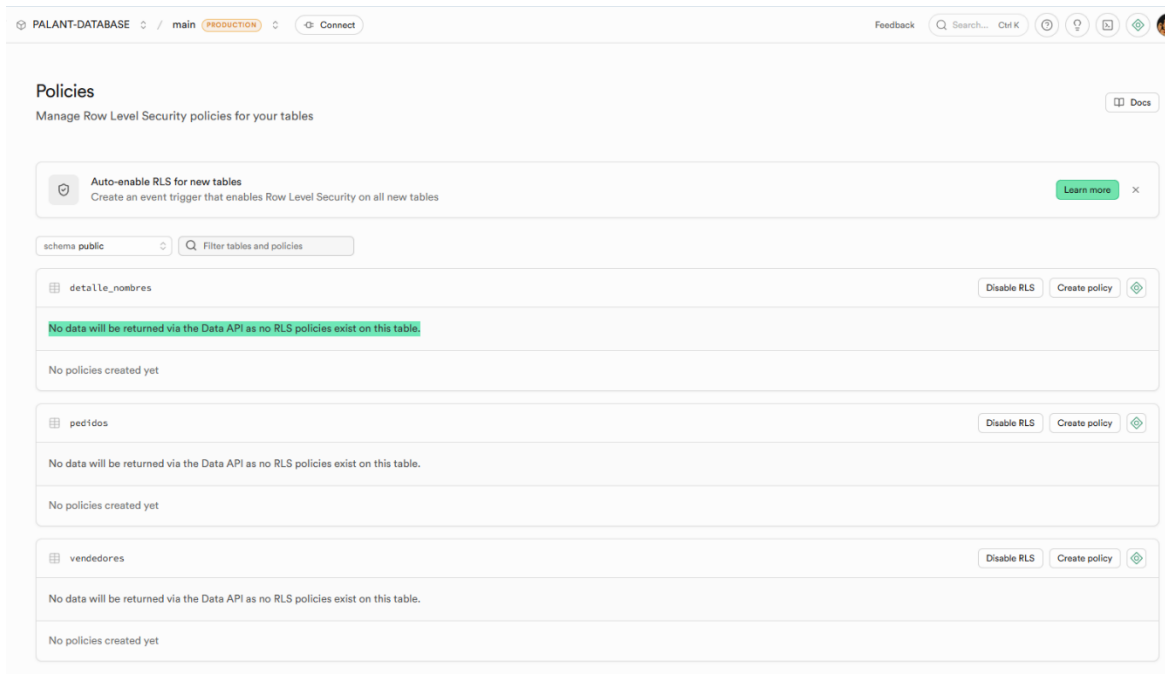
Para garantizar la calidad y confiabilidad del software en un entorno de producción exigente como el taller textil, el desarrollo se respaldó bajo un enfoque TDD. Mediante el uso de herramientas de prueba automatizadas como **JUnit Jupiter** y **Mockito** (configuradas a través de Apache Maven), se diseñaron y ejecutaron pruebas unitarias sobre los servicios y componentes críticos antes y durante la codificación, reduciendo de manera significativa los posibles errores en producción y asegurando que las especificaciones de tallas y pedidos fuesen precisas.



Captura evidenciando el uso de dependencias para el testeo del sistema.

C. Consideraciones de seguridad

Sabiendo que la base de datos se aloja en la nube, la arquitectura contempló múltiples capas de seguridad. En el backend, se aprovechó la funcionalidad de Row Level Security (RLS) de PostgreSQL en Supabase, permitiendo aislar y proteger la información a nivel de fila. En la capa de aplicación, el sistema asegura que las contraseñas de los usuarios no viajen ni se almacenen en texto plano, aplicando técnicas de hashing criptográfico. Además, los roles definidos (Vendedor, Diseñador, Jefe de Taller) restringen estrictamente el acceso a los módulos según el nivel de autorización del usuario que inicia sesión.



"Como se aprecia en la captura, al habilitar la funcionalidad RLS en las tablas, el sistema restringe inmediatamente el flujo de datos (nadie puede leer ni escribir datos libremente). Esto garantiza que la información de los pedidos sea confidencial y que el servidor backend (o las políticas que se escriban después) sea el único con autoridad para extraer los datos.

4.2. Desarrollo iterativo

A. Avances del desarrollo

En consonancia con la metodología híbrida y el componente ágil detallado en la planificación, el ciclo de vida de PALANT-PEDIDOS siguió un proceso iterativo. El desarrollo no esperó a tener un producto final y estático, sino que se dividió en incrementos funcionales (ej. Módulo Administrativo, Panel Kanban, Módulo de Diseño Vectorial). A través de herramientas de gestión como Trello/Jira, se trazaron historias de usuario que fueron completadas en ciclos semanales, permitiendo observar un avance continuo y ordenado en el código fuente de la aplicación.

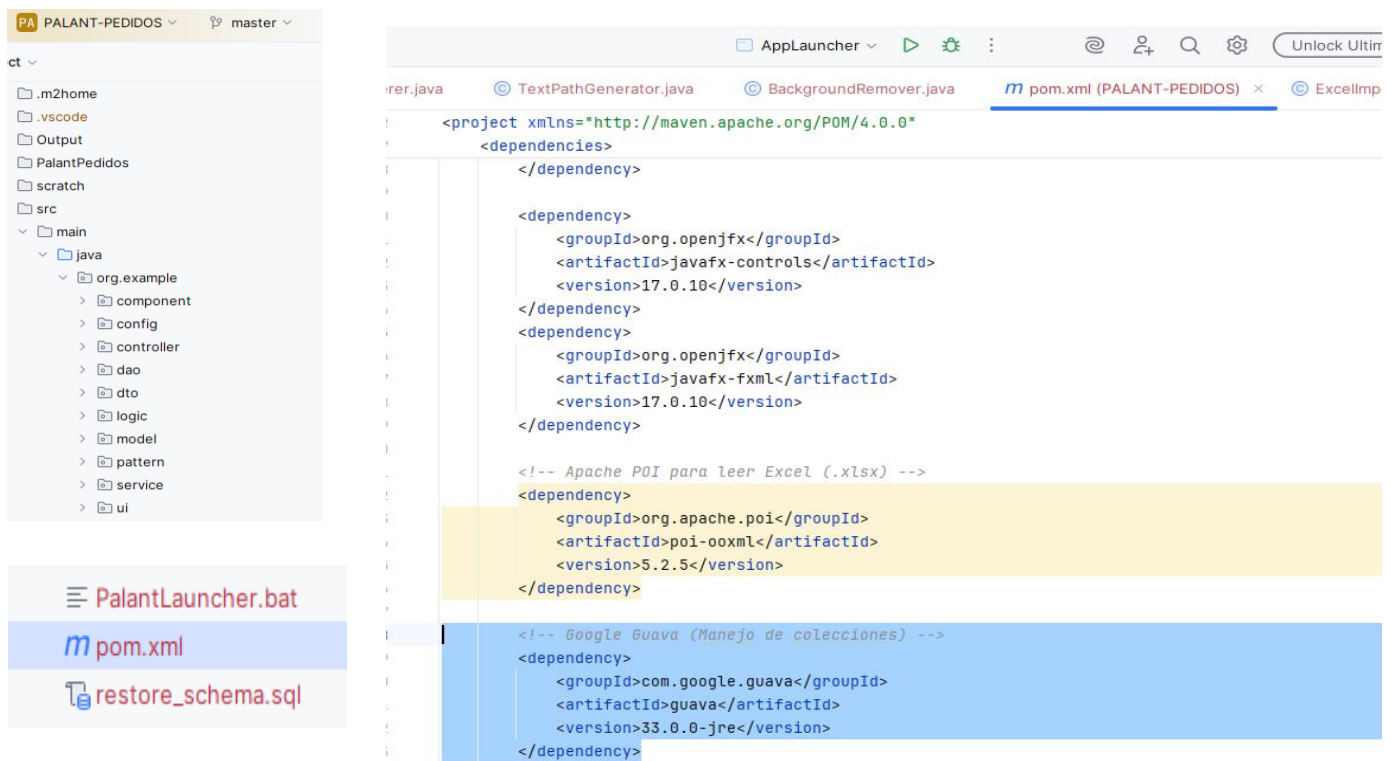
B. Validación y retroalimentación

Uno de los riesgos principales identificados fue el posible rechazo tecnológico por parte del personal de taller. Para mitigarlo, cada iteración funcional incluyó un proceso de validación directo con los usuarios finales (diseñadores y costureros). Al entregarles prototipos funcionales tempranos de las interfaces en JavaFX, se logró obtener una retroalimentación valiosa, simplificando botones, haciendo los campos obligatorios más visibles y adaptando el panel Kanban a su flujo de trabajo real.

4.3. Uso de librerías Java

Para no "reinventar la rueda" y acelerar el tiempo de desarrollo, el proyecto incorporó un ecosistema de librerías de soporte técnico declaradas de forma centralizada en el pom.xml:

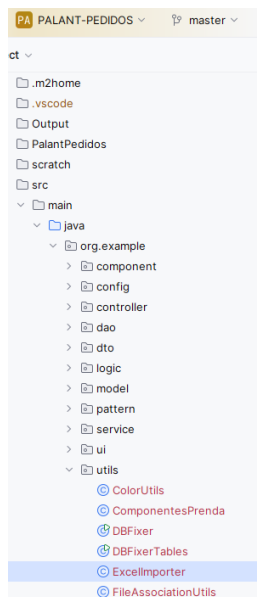
- **Google Guava:** Implementado para potenciar y facilitar la manipulación de colecciones y estructuras de datos complejas generadas en los módulos de diseño.



1. Agrupación compleja de datos (Multimapas): En una orden de ropa deportiva, un cliente pide varias camisetas, pero agrupadas por tallas (ej. 5 talla S, 10 talla M, 2 talla XL). En Java normal, agrupar esto es tedioso y requiere mucho código. Con Google Guava, se usan estructuras avanzadas como el Multimap, que permite agrupar todas las tallas y nombres de los jugadores de un pedido en una sola estructura de datos de forma automática y muy eficiente.
2. Caché en memoria (Optimización): Al usar una base de datos en la nube (Supabase), no es eficiente pedirle la misma lista de clientes o los mismos estados de las órdenes al servidor a cada segundo (porque puede causar lentitud). Guava tiene una herramienta llamada CacheBuilder que guarda temporalmente en la memoria de la computadora los datos que más se usan. Si el internet falla un microsegundo o está lento, el sistema carga los datos desde la caché de Guava, haciendo que tu aplicación JavaFX se sienta súper rápida y fluida para el operario del taller.
3. Prevención de Errores Críticos (NullPointerExceptions): Al escribir los nombres que van en la espalda de las camisetas (Dorsales), los operarios pueden dejar espacios en blanco o nulos por accidente. Guava tiene utilidades como `Strings.isNullOrEmpty()` o la clase `Optional` que filtran y limpian textos antes de mandarlos a la base de datos o al diseño final, evitando que el programa se "crashee" o se cierre de golpe en medio del taller.

```
nderer.java  TextPathGenerator.java  BackgroundRemover.java  pom.xml (PALANT-PEDIDOS)  GuavaOrderUtil.java  ExcelImporter.java
4      import com.google.common.collect.Multimap;
5      import org.example.dto.save.PrendaStateDTO;
6
7      import java.util.List;
8      import java.util.Collection;
9
10     /**
11     * Utilidad basada en Google Guava para la agrupación avanzada de órdenes y prendas.
12     * Implementado para optimizar el rendimiento del panel Kanban en el área de Taller.
13     */
14     public class GuavaOrderUtil { no usages
15
16         /**
17         * Agrupa una lista plana de prendas según su talla de forma automática
18         * utilizando un Multimap de Google Guava.
19         */
20         @param prendas Lista completa de prendas de un pedido
21         @return Multimap que asocia cada talla con sus respectivas prendas
22         */
23         public static Multimap<String, PrendaStateDTO> agruparPrendasPorTalla(List<PrendaStateDTO> prendas) { no usages
24             Multimap<String, PrendaStateDTO> prendasAgrupadas = ArrayListMultimap.create();
25
26             if (prendas != null) {
27                 for (PrendaStateDTO prenda : prendas) {
28                     // Si la talla es nula, la agrupa en la categoría "Desconocida"
29                     String talla = (prenda.getTalla() != null && !prenda.getTalla().isEmpty())
30                         ? prenda.getTalla() : "Desconocida";
31
32                     prendasAgrupadas.put(talla, prenda);
33                 }
34             }
35
36             return prendasAgrupadas;
37         }
38
39         /**
40         * Obtiene rápidamente todas las prendas asociadas a una talla específica.
41         */
42         public static Collection<PrendaStateDTO> obtenerPrendasDeTalla(Multimap<String, PrendaStateDTO> mapa, String talla) { no usages
43             return mapa.get(talla);
44         }
45     }
```

- **Apache POI:** Librería fundamental para la integración ofimática, permitiendo la lectura, generación e interoperabilidad de reportes documentales en formato Microsoft Excel (archivos .xlsx).

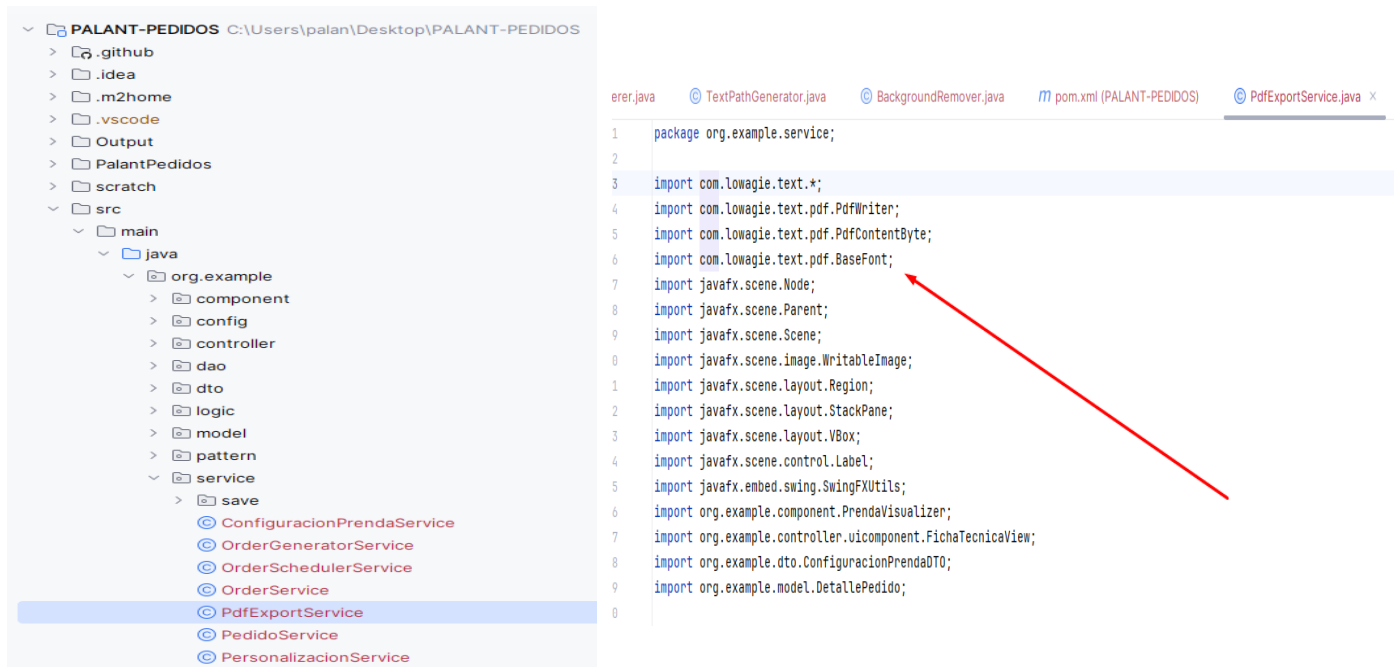


```
package org.example.utils;

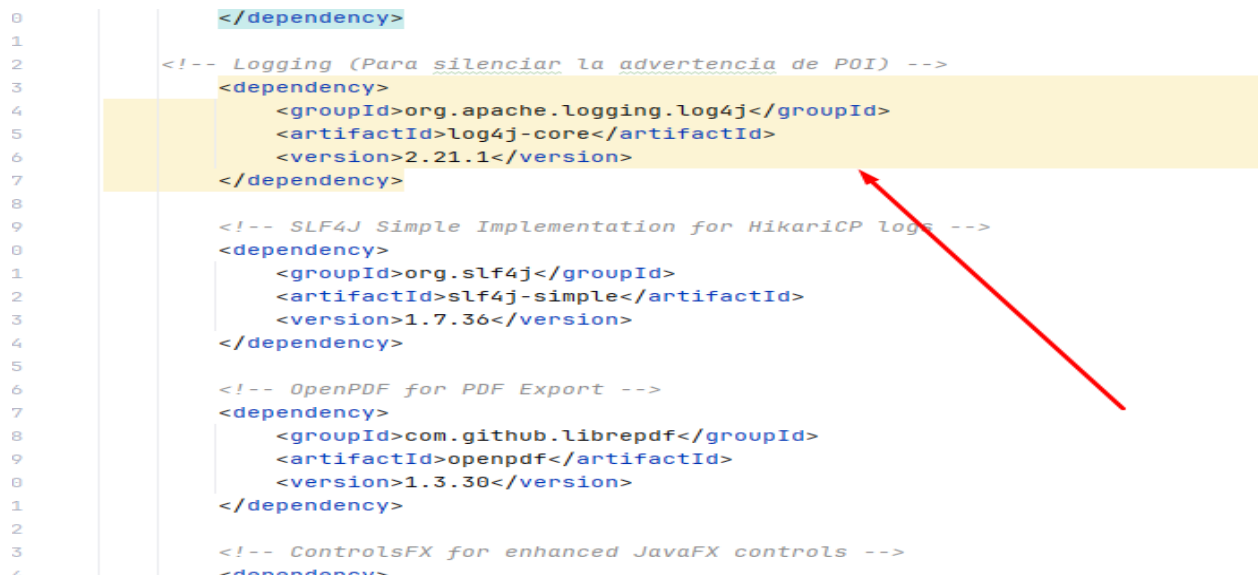
import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
import org.example.model.DetallePedido;

import java.io.File;
import java.io.FileInputStream;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
```

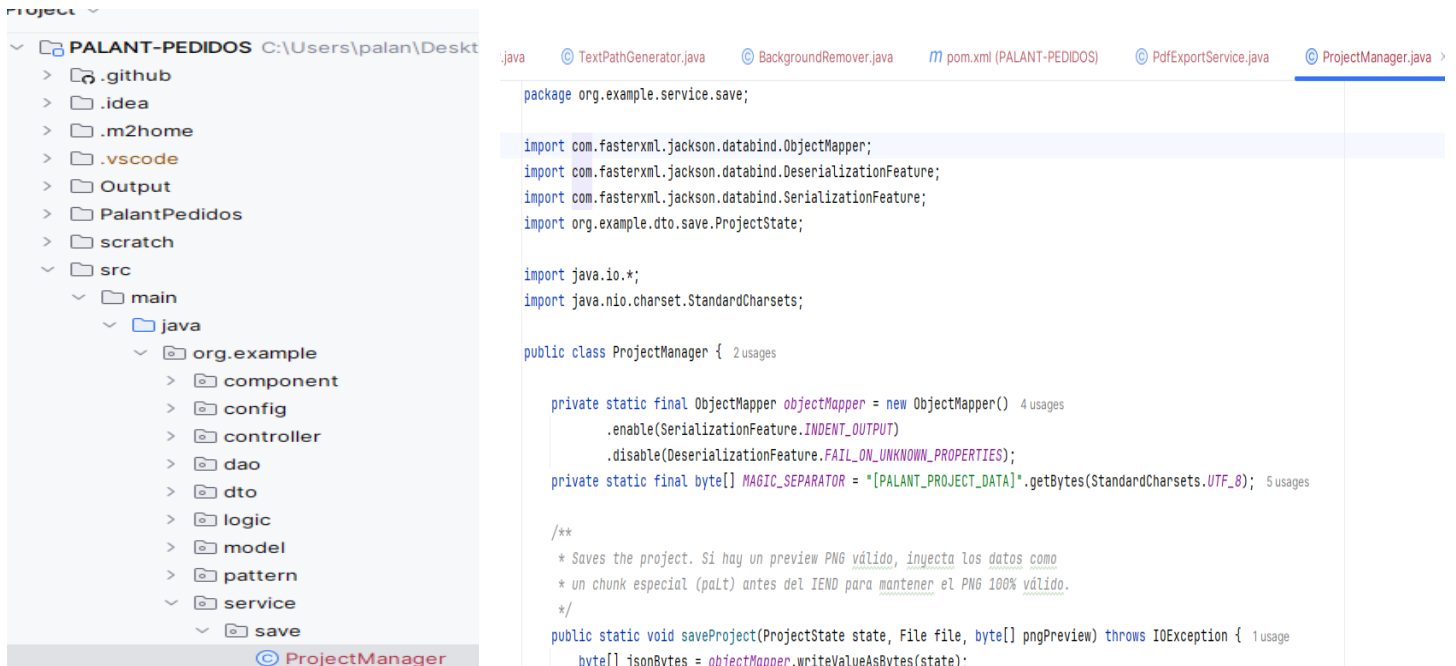
- **OpenPDF:** Seleccionado para la renderización visual y exportación automatizada de los órdenes a formato PDF (Fichas Técnicas Digitales).



- **Log4j / SLF4J:** Integradas para el registro estructurado de eventos en la aplicación (logs), permitiendo a los administradores técnicos monitorear posibles fallos o advertencias de conexiones durante el tiempo de ejecución.



- **Jackson:** Utilizado para manejar serialización y deserialización de archivos y datos en formato JSON, clave para la comunicación con la API de Supabase.

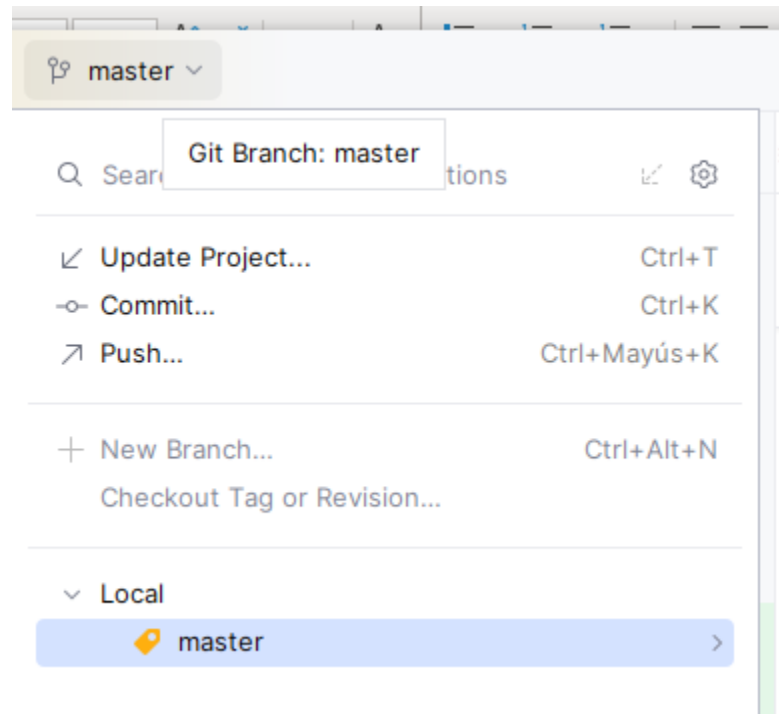
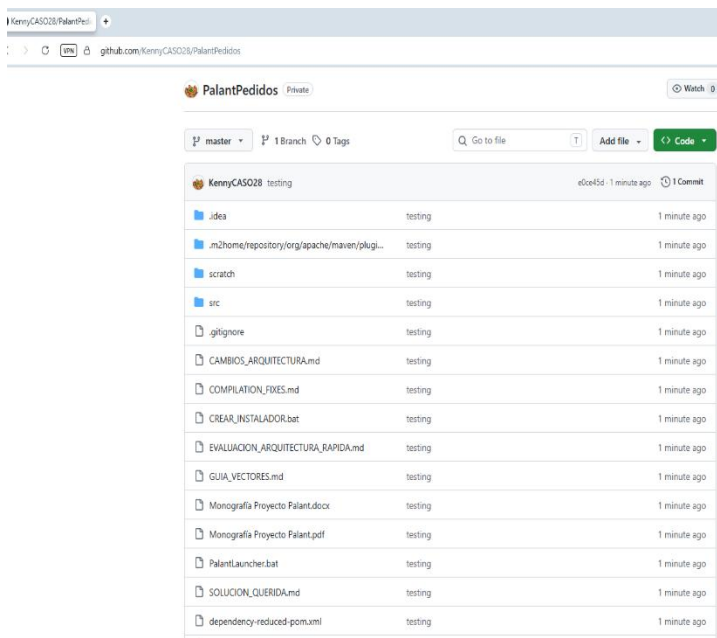


4.4. Control de versiones

Estrategia de ramificación e Integración con GitHub El almacenamiento seguro del código fuente y el historial de cambios se gestionó utilizando el sistema distribuido Git. Para organizar las entregas del equipo técnico, se aplicó una estrategia de ramificación basada en características (Feature Branching). Esto permitió que módulos experimentales (como la implementación de la matemática gráfica para el editor vectorial) fueran desarrollados y probados en ramas aisladas sin afectar la rama principal (main o master) que contiene el código estable del proyecto.

Posteriormente, el repositorio local se integró en la nube a través de la plataforma GitHub, lo que brindó un respaldo de contingencia absoluto ante cualquier fallo de hardware en las computadoras de desarrollo y habilitó un entorno colaborativo profesional.

Buenas prácticas Para la trazabilidad de los avances, se estandarizó el uso de mensajes de commit atómicos y descriptivos, asegurando que cada modificación en el código dejara un registro claro y auditable.



<https://github.com/KennyCASO28/PalantPedidos>

Referencias:

Rahman, M. (2024). Agile Project Management: Analyzing the effectiveness of agile methodologies in IT projects compared to traditional approaches. *Academic Journal on Business Administration Innovation & Sustainability*, 4(4), 53-69.

Silva, A., Costa, C., & Pereira, L. (2026). Quality, security, and privacy assurance in software development: proactive integration or just workflow-slowing checkpoints?. *International Journal of Information Systems and Project Management*, 14(1), 1-20.

Sarmiento Príncipe, R. M. (2024). *Desarrollo de un aplicativo web basado en metodología ágil y su impacto en la gestión de información académica* [Tesis de pregrado, Universidad de Huánuco]. Repositorio Institucional UDH.

Zamora, V. J. (2023). *Implementación de un sistema de planificación de recursos empresariales (ERP) utilizando metodologías ágiles para la optimización de procesos* [Tesis de maestría, Universidad Peruana de Ciencias Aplicadas]. Repositorio Académico UPC.